

Bootstrap 5 Get Started

Setting up **Bootstrap 5** is the quickest way to build responsive, mobile-first websites. Since it's a CSS framework, you don't need to "install" it in the traditional sense; you just need to link the files to your HTML.

Here is the most efficient way to get up and running.

1. The Quick Start (CDN)

The fastest way to start is by using the **Content Delivery Network (CDN)**. This pulls the Bootstrap files directly from a server rather than requiring you to download them.

Copy-Paste Starter Template

Create an `index.html` file and paste the following code:

HTML

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Bootstrap 5 Demo</title>
    <link
href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css
" rel="stylesheet">
    </head>
    <body>
      <div class="container mt-5">
        <h1>Hello, Bootstrap 5!</h1>
        <button class="btn btn-primary">This is a Bootstrap Button</button>
      </div>

      <script
src="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.mi
n.js"></script>
    </body>
  </html>
```

2. Key Features to Know

Bootstrap 5 has moved away from jQuery, making it lighter and faster. Here are the three pillars you'll use most:

- **The Grid System:** A flexbox-based layout that uses a 12-column system.
 - **Components:** Pre-designed UI elements like navbars, cards, modals, and buttons.
 - **Utilities:** Helper classes for spacing (`mt-5` for margin-top), colors (`text-primary`), and alignment.
-

3. The Layout Structure

To make Bootstrap work correctly, you must wrap your content in a **container**.

Class	Purpose
<code>.container</code>	Fixed width based on the screen size (centered).
<code>.container-fluid</code>	Full width, spanning the entire viewport.
<code>.row</code>	A horizontal wrapper for columns.
<code>.col</code>	The actual content holders within a row.

Export to Sheets

4. Alternative: Package Managers

If you are working on a professional project using **Node.js**, you should install it via NPM to have better control over updates and customization.

Run this in your terminal:

```
Bash
npm install bootstrap
```

Note: If you install via NPM, you will need to manually import the CSS and JS files into your main entry point (like `index.js` or `app.scss`).

Pro-Tip: Icons

Bootstrap 5 doesn't include icons by default anymore. If you want those slick arrows or house icons, you'll want to add **Bootstrap Icons**:

HTML

```
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.0/font/bootstrap-icons.css">
```

Are you looking to build a specific component first, like a navigation bar or a landing page hero section?

Bootstrap 5 Containers

In Bootstrap 5, **Containers** are the most basic layout element and are **required** when using the default grid system. They are used to contain, pad, and (sometimes) center the content within them.

1. How Containers Work

Containers act as a wrapper for your site's content. They provide margins and padding that change based on the user's screen size (breakpoints).

- **Fixed Width:** Uses specific widths for different screen sizes (e.g., 1140px on desktops).
 - **Fluid Width:** Always stays at 100% width, regardless of screen size.
 - **Responsive:** Stays at 100% width until it reaches a specific breakpoint.
-

2. Container Types

The Default Container

The `.container` class is a responsive, fixed-width container. It centers your content and sets a `max-width` at each responsive breakpoint.

HTML

```
<div class="container">  
  </div>
```

Fluid Container

The `.container-fluid` class is for a full-width container, spanning the entire width of the viewport.

HTML

```
<div class="container-fluid">  
  </div>
```

Responsive Containers

These allow you to specify a container that is 100% wide until a specific breakpoint is reached. For example, `.container-md` will be 100% wide on small screens but will become a fixed-width container once the screen size is "Medium" or larger.

Class	Extra Small (<576px)	Small (≥576px)	Medium (≥768px)	Large (≥960px)
<code>.container</code>	100%	540px	720px	960px
<code>.container-sm</code>	100%	540px	720px	960px
<code>.container-md</code>	100%	100%	720px	960px
<code>.container-lg</code>	100%	100%	100%	960px
Export to Sheets				

3. Best Practices

- **Don't Nest Containers:** You should generally avoid putting a `.container` inside another `.container`. It can cause double padding and alignment issues.
 - **Direct Child of Row:** In the grid system, a `.row` should be a direct child of a `.container`, and `.col` classes should be direct children of the `.row`.
 - **Center Alignment:** Containers are automatically centered in the browser because they have `margin-right: auto` and `margin-left: auto` applied.
-

4. Padding and Gutters

By default, containers have **15px** of horizontal padding. This matches the "gutters" (spacing) between columns in the grid system, ensuring that your text aligns perfectly with the rest of your layout.

Do you want to see how to combine these containers with the **Grid System** (Rows and Columns) to create a layout?

Bootstrap 5 Grids

The **Bootstrap 5 Grid System** is a powerful, mobile-first flexbox system used to build layouts of all shapes and sizes. It uses a **12-column system**, which means every row is divided into 12 equal parts.

1. The Grid Structure

To use the grid, you follow a strict three-step hierarchy:

1. **Container:** Wraps the entire grid (as discussed previously).
2. **Row:** Acts as a wrapper for columns (clears the container padding).
3. **Columns:** The actual content holders that sit inside rows.

Basic Example

HTML

```
<div class="container">
  <div class="row">
    <div class="col">Column 1 of 3</div>
    <div class="col">Column 2 of 3</div>
    <div class="col">Column 3 of 3</div>
  </div>
</div>
```

In this example, Bootstrap automatically divides the space into 3 equal parts.

2. Setting Column Widths

Since the grid is based on 12 units, you can specify how much space a column takes using the `.col-X` class, where **X** is a number from 1 to 12.

- `.col-4`: Takes up 1/3 of the row ($12/4=3$).
- `.col-6`: Takes up half the row ($12/6=2$).
- `.col-12`: Takes up the full width.

HTML

```
<div class="row">
  <div class="col-8">Main Content (wider)</div>
  <div class="col-4">Sidebar (narrower)</div>
</div>
```

3. Responsive Breakpoints

One of the best features of the grid is the ability to change column sizes based on the screen size. Bootstrap uses "infixes" to handle this:

Infix	Screen Size	Description
(None)	<576px	Extra Small (Phones)
sm	≥576px	Small (Large Phones)
md	≥768px	Medium (Tablets)
lg	≥992px	Large (Desktops)
xl	≥1200px	Extra Large (Large Desktops)

Export to Sheets

Responsive Example

If you want a column to be full-width on mobile but half-width on tablets, you would use:

HTML

```
<div class="col-12 col-md-6">  
  This is stacked on mobile and side-by-side on tablets.  
</div>
```

4. Key Grid Utilities

- **Equal-width:** Use just `.col` to let Bootstrap handle the math automatically.
 - **Variable width content:** Use `.col-auto` to make the column as wide as its content.
 - **Gutters:** These are the gaps between columns. You can change them using `.g-*` classes (e.g., `.g-3` for standard spacing or `.g-0` to remove them entirely).
 - **Alignment:** Use flexbox utilities on the `.row` to align columns (e.g., `.align-items-center` or `.justify-content-center`).
-

Grid Challenge

Try creating a "Pricing Section" layout. You'll need:

1. A **Container**.
2. A **Row**.

Three **Columns** that are full-width on mobile (`col-12`) but take up equal space on Bootstrap 5 Text/Typography

In Bootstrap 5, **Typography** is designed to be clean, accessible, and responsive. It uses a "native font stack" that automatically selects the best font for your operating system (like San Francisco for iOS or Segoe UI for Windows).

1. Headings

Bootstrap styling applies to standard `<h1>` through `<h6>` tags. However, if you want to make a paragraph look like a heading without using the actual HTML tag, you can use heading classes.

- **Standard Headings:** `<h1>` to `<h6>`
- **Heading Classes:** `.h1` to `.h6` (useful for maintaining semantic structure while changing visual size).

Display Headings

When you need a heading to really stand out (like on a landing page), use **Display headings**. These are larger and more "opinionated" than standard headings.

HTML

```
<h1 class="display-1">Display 1</h1>
<h1 class="display-2">Display 2</h1>
<h1 class="display-3">Display 3</h1>
<h1 class="display-4">Display 4</h1>
<h1 class="display-5">Display 5</h1>
<h1 class="display-6">Display 6</h1>
```

2. Lead and Body Text

Standard body text has a default `font-size` and `line-height`. To make a paragraph stand out (like an introductory summary), use the `.lead` class.

HTML

```
<p class="lead"> This is a lead paragraph. It stands out from regular
text.</p>
<p> This is standard body text for comparison.</p>
```

3. Inline Text Elements

Bootstrap provides classes to style traditional HTML inline tags quickly:

- **.mark:** Highlights text (mimics `<mark>`).
 - **.small:** Decreases font size to 80% (mimics `<small>`).
 - **.text-decoration-none:** Removes underlines (great for links).
 - **.text-lowercase / .text-uppercase / .text-capitalize:** Quickly changes text casing.
-

4. Text Alignment and Wrapping

You can align text responsively using utility classes. This is extremely helpful for mobile design.

Class	Result
<code>.text-start</code>	Left aligned (LTR)
<code>.text-center</code>	Centered
<code>.text-end</code>	Right aligned (LTR)
<code>.text-nowrap</code>	Prevents text from wrapping to a new line

[Export to Sheets](#)

Responsive Alignment Example: `<p class="text-center text-md-start">Center on mobile, left-aligned on tablet/desktop.</p>`

5. Text Colors (Utilities)

Bootstrap includes several contextual classes to change the color of your text.

HTML

```
<p class="text-primary">Blue text (Primary)</p>
<p class="text-secondary">Grey text (Secondary)</p>
<p class="text-success">Green text (Success)</p>
<p class="text-danger">Red text (Danger)</p>
<p class="text-muted">Faded grey text</p>
```

6. Blockquotes

For quoting blocks of content from another source within your document, use the `.blockquote` class.

HTML

```
<figure>
  <blockquote class="blockquote">
    <p>A well-known quote, contained in a blockquote element.</p>
  </blockquote>
  <figcaption class="blockquote-footer">
    Someone famous in <cite title="Source Title">Source Title</cite>
  </figcaption>
</figure>
```

medium screens and up (`col-md-4`). Bootstrap 5 Colors

In Bootstrap 5, colors are handled through **utility classes** that apply to text, backgrounds, and borders. These colors are "contextual," meaning they are named based on their intent (e.g., "success" for positive actions) rather than just the color name.

1. The Core Color Palette

Bootstrap uses a standard set of colors that remain consistent across all components.

Purpose	Color	Class Suffix
Primary	Blue	-primary
Secondary	Grey	-secondary
Success	Green	-success
Danger	Red	-danger
Warning	Yellow	-warning
Info	Cyan	-info
Light	Off-white	-light
Dark	Near-black	-dark
Export to Sheets		

2. Text Colors

To change the color of your text, use the `.text-{color}` utility.

HTML

```
<p class="text-primary">This is primary blue text.</p>
<p class="text-success">This is success green text.</p>
<p class="text-danger">This is danger red text.</p>
<p class="text-muted">This is muted (grey) text.</p>
<p class="text-white bg-dark">This is white text on a dark background.</p>
```

Opacity

Bootstrap 5.1+ allows you to change the **opacity** of text using `.text-opacity-{value}` (values: 25, 50, 75). *Example:* `<p class="text-primary text-opacity-50">Faded blue text</p>`

3. Background Colors

To change the background of an element, use the `.bg-{color}` utility.

HTML

```
<div class="p-3 mb-2 bg-primary text-white">.bg-primary</div>
<div class="p-3 mb-2 bg-warning text-dark">.bg-warning</div>
```

Background Gradients

If you want a subtle vertical gradient instead of a flat color, add the `.bg-gradient` class.

HTML

```
<div class="p-3 bg-success bg-gradient text-white">Success with
Gradient</div>
```

4. Background Subtlety (Subtle Colors)

Bootstrap 5.3 introduced **subtle** background and border colors, which are perfect for alerts or light-themed UI elements. These are much lighter than the standard palette.

- **Class:** `.bg-primary-subtle`, `.bg-danger-subtle`, etc.

[Image comparison of Bootstrap 5 standard background colors versus subtle background colors]

5. Border Colors

You can also apply these colors to borders. Note that you must first add the `.border` class to make the border visible.

HTML

```
<div class="border border-primary">Blue Border</div>
<div class="border border-success border-3">Thick Green Border</div>
```

6. Links and Buttons

Colors also apply to interactive elements:

- **Links:** `.link-primary`, `.link-success` (these include hover states).
 - **Buttons:** `.btn-primary`, `.btn-outline-danger`.
-

Practical Tip: Contrast

When using background colors, always ensure your text color is readable. Use `.text-white` or `.text-dark` to create high contrast.

For example, **Warning** (yellow) usually looks best with `.text-dark`, while **Primary** (blue) or **Danger** (red) almost always requires `.text-white`.

Bootstrap 5 Tables

In Bootstrap 5, tables are designed to be clean and legible. By default, a standard HTML `<table>` has almost no styling. Adding the `.table` class instantly applies light padding and horizontal dividers.

1. Basic Table Structure

To get started, simply add the `.table` class to your `<table>` tag.

HTML

```
<table class="table">
  <thead>
    <tr>
      <th scope="col">#</th>
      <th scope="col">First</th>
      <th scope="col">Last</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">1</th>
      <td>Mark</td>
      <td>Otto</td>
    </tr>
  </tbody>
</table>
```

2. Table Variations

You can combine multiple classes to change the look and feel of your data.

Class	Effect
<code>.table-striped</code>	Adds "zebra-stripping" (alternating grey/white rows).
<code>.table-hover</code>	Highlights a row when the mouse pointer hovers over it.

Class	Effect
<code>.table-bordered</code>	Adds borders on all sides of the table and cells.
<code>.table-borderless</code>	Removes all borders from the table.
<code>.table-sm</code>	Cuts cell padding in half to make the table more compact.
Export to Sheets	

3. Table Colors

You can apply contextual colors to the entire table, specific rows, or even individual cells using the `.table-{color}` classes.

- **Dark Table:** Use `.table-dark` for a black background with white text.
- **Colored Rows:** Use `.table-primary`, `.table-success`, `.table-danger`, etc.

HTML

```
<tr class="table-success">
  <td>This row is green to indicate a successful status.</td>
</tr>
```

4. Table Head Options

Sometimes you want the header to stand out from the rest of the body. You can style the `<thead>` specifically.

- **Light Head:** `<thead class="table-light">`
 - **Dark Head:** `<thead class="table-dark">`
-

5. Responsive Tables

Tables can often break a mobile layout because they are naturally wide. To fix this, wrap your `<table>` in a `<div>` with the `.table-responsive` class.

HTML

```
<div class="table-responsive">
  <table class="table">
    ...
  </table>
</div>
```

This adds a horizontal scrollbar on small screens while keeping the rest of your page layout intact.

6. Vertical Alignment

If your cells have varying amounts of content, you can align the text vertically using:

- `.align-top`
- `.align-middle` (common for professional dashboards)
- `.align-bottom`

Bootstrap 5 Images

In Bootstrap 5, styling images is primarily about making them **responsive** so they don't overflow their parent containers, and adding quick decorative touches like rounded corners or shadows.

1. Responsive Images

By default, images in HTML are not responsive. If an image is 1000px wide and the screen is only 400px wide, the image will stick out.

To fix this, apply the `.img-fluid` class. This sets `max-width: 100%` and `height: auto`.

HTML

```

```

2. Image Thumbnails

If you want to give an image a slight rounded 1px border appearance (common for profile pictures or gallery previews), use the `.img-thumbnail` class.

HTML

```

```

3. Aligning Images

Images are inline elements by default. To align them, you can use float utilities or text alignment on the parent container.

Using Floats

- `.float-start`: Aligns the image to the left.
- `.float-end`: Aligns the image to the right.

Using Centering

To center an image, you must make it a block element using `.d-block` and then use horizontal auto-margins with `.mx-auto`.

HTML

```

```

4. Rounded Corners

Since Bootstrap 5 provides utility classes for borders, you can easily change the shape of your images without writing custom CSS.

Class	Effect
<code>.rounded</code>	Small rounded corners on all sides.
<code>.rounded-circle</code>	Turns the image into a perfect circle (best for square images).
<code>.rounded-pill</code>	Creates a pill-like shape.
<code>.rounded-top</code>	Only rounds the top two corners.

[Export to Sheets](#)

5. The `<picture>` Element

If you are using the HTML5 `<picture>` element to serve different images for different screen sizes, make sure to apply the `.img-fluid` class to the `<img>` tag inside the picture element, not the `<picture>` tag itself.

HTML

```
<picture>  
  <source srcset="..." type="image/svg+xml">  
    
</picture>
```

6. Figures

If you need to display an image with a caption, Bootstrap uses the `<figure>` and `<figcaption>` elements paired with specific classes.

HTML

```
<figure class="figure">
  
  <figcaption class="figure-caption text-end">A beautiful morning in the
  forest.</figcaption>
</figure>
```

Bootstrap 5 Jumbotron

In **Bootstrap 5**, the specific `.jumbotron` class from version 4 has been **removed**.

Instead of a single dedicated class, Bootstrap 5 encourages you to use a combination of **Utility Classes** (like spacing, colors, and borders) to create a "Jumbotron" effect. This gives you more flexibility to customize the look.

1. Creating a Basic Jumbotron

To recreate the classic Jumbotron look, you typically use a `<div>` with background color, padding, and rounded corners.

HTML

```
<div class="p-5 mb-4 bg-light rounded-3">
  <div class="container-fluid py-5">
    <h1 class="display-5 fw-bold">Custom Jumbotron</h1>
    <p class="col-md-8 fs-4">Using a series of utilities, you can create this
    jumbotron, just like in previous versions of Bootstrap.</p>
    <button class="btn btn-primary btn-lg" type="button">Example
    button</button>
  </div>
</div>
```

2. Key Classes Used

When building your own Jumbotron, these are the "building blocks" you'll use:

Class	Purpose
<code>.p-5</code>	Adds large padding inside the box.
<code>.mb-4</code>	Adds a margin at the bottom to separate it from content.
<code>.bg-light</code>	Sets a light grey background (or use <code>.bg-dark</code> , <code>.bg-primary</code>).
<code>.rounded-3</code>	Adds rounded corners to the box.

Class	Purpose
<code>.display-5</code>	Makes the heading large and prominent.
<code>.fw-bold</code>	Makes the text bold.
<code>.fs-4</code>	Sets a specific font size for the paragraph.

Export to Sheets

3. Dark Mode Jumbotron

You can easily switch the theme by swapping the background and text color utilities.

HTML

```
<div class="p-5 text-white bg-dark rounded-3">
  <h2>Dark Mode Hero</h2>
  <p>This version uses dark background utilities for a high-contrast
look.</p>
  <hr class="my-4">
  <p>It uses utility classes for typography and spacing to match your
brand.</p>
  <button class="btn btn-outline-light" type="button">Learn More</button>
</div>
```

4. Full-Width (Fluid) Jumbotron

If you want the Jumbotron to span the entire width of the screen without rounded corners, remove the `.container` or `.rounded-3` classes.

HTML

```
<div class="p-5 bg-primary text-white">
  <div class="container">
    <h1 class="display-4">Full Width Hero</h1>
    <p class="lead">This spreads across the entire viewport.</p>
  </div>
</div>
```

Summary: Why did it change?

Bootstrap 4's `.jumbotron` was a rigid component. By moving to **Utility Classes** in Version 5:

- **Less CSS:** The framework is lighter because it doesn't need a specific "Jumbotron" style definition.
- **Total Control:** You can decide exactly how much padding, what color, and how rounded you want the box to be without fighting default styles.

Bootstrap 5 Get Started

Setting up **Bootstrap 5** is the quickest way to build responsive, mobile-first websites. Since it's a CSS framework, you don't need to "install" it in the traditional sense; you just need to link the files to your HTML.

Here is the most efficient way to get up and running.

1. The Quick Start (CDN)

The fastest way to start is by using the **Content Delivery Network (CDN)**. This pulls the Bootstrap files directly from a server rather than requiring you to download them.

Copy-Paste Starter Template

Create an `index.html` file and paste the following code:

HTML

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Bootstrap 5 Demo</title>
    <link
href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css
" rel="stylesheet">
  </head>
  <body>
    <div class="container mt-5">
      <h1>Hello, Bootstrap 5!</h1>
      <button class="btn btn-primary">This is a Bootstrap Button</button>
    </div>

    <script
src="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.mi
n.js"></script>
  </body>
</html>
```

2. Key Features to Know

Bootstrap 5 has moved away from jQuery, making it lighter and faster. Here are the three pillars you'll use most:

- **The Grid System:** A flexbox-based layout that uses a 12-column system.
- **Components:** Pre-designed UI elements like navbars, cards, modals, and buttons.
- **Utilities:** Helper classes for spacing (`mt-5` for margin-top), colors (`text-primary`), and alignment.

3. The Layout Structure

To make Bootstrap work correctly, you must wrap your content in a **container**.

Class	Purpose
<code>.container</code>	Fixed width based on the screen size (centered).
<code>.container-fluid</code>	Full width, spanning the entire viewport.
<code>.row</code>	A horizontal wrapper for columns.
<code>.col</code>	The actual content holders within a row.

Export to Sheets

4. Alternative: Package Managers

If you are working on a professional project using **Node.js**, you should install it via NPM to have better control over updates and customization.

Run this in your terminal:

Bash
`npm install bootstrap`

Note: If you install via NPM, you will need to manually import the CSS and JS files into your main entry point (like `index.js` or `app.scss`).

Pro-Tip: Icons

Bootstrap 5 doesn't include icons by default anymore. If you want those slick arrows or house icons, you'll want to add **Bootstrap Icons**:

HTML

```
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.0/font/bootstrap-icons.css">
```

Are you looking to build a specific component first, like a navigation bar or a landing page hero section?

You said
Bootstrap 5 Containers

In Bootstrap 5, **Containers** are the most basic layout element and are **required** when using the default grid system. They are used to contain, pad, and (sometimes) center the content within them.

1. How Containers Work

Containers act as a wrapper for your site's content. They provide margins and padding that change based on the user's screen size (breakpoints).

- **Fixed Width:** Uses specific widths for different screen sizes (e.g., 1140px on desktops).
- **Fluid Width:** Always stays at 100% width, regardless of screen size.
- **Responsive:** Stays at 100% width until it reaches a specific breakpoint.

2. Container Types

The Default Container

The `.container` class is a responsive, fixed-width container. It centers your content and sets a `max-width` at each responsive breakpoint.

HTML

```
<div class="container">  
  </div>
```

Fluid Container

The `.container-fluid` class is for a full-width container, spanning the entire width of the viewport.

HTML

```
<div class="container-fluid">  
  </div>
```

Responsive Containers

These allow you to specify a container that is 100% wide until a specific breakpoint is reached. For example, `.container-md` will be 100% wide on small screens but will become a fixed-width container once the screen size is "Medium" or larger.

Class	Extra Small (<576px)	Small (≥576px)	Medium (≥768px)	Large (≥960px)
<code>.container</code>	100%	540px	720px	960px
<code>.container-sm</code>	100%	540px	720px	960px
<code>.container-md</code>	100%	100%	720px	960px
<code>.container-lg</code>	100%	100%	100%	960px

Export to Sheets

3. Best Practices

- **Don't Nest Containers:** You should generally avoid putting a `.container` inside another `.container`. It can cause double padding and alignment issues.
 - **Direct Child of Row:** In the grid system, a `.row` should be a direct child of a `.container`, and `.col` classes should be direct children of the `.row`.
 - **Center Alignment:** Containers are automatically centered in the browser because they have `margin-right: auto` and `margin-left: auto` applied.
-

4. Padding and Gutters

By default, containers have **15px** of horizontal padding. This matches the "gutters" (spacing) between columns in the grid system, ensuring that your text aligns perfectly with the rest of your layout.

Do you want to see how to combine these containers with the **Grid System** (Rows and Columns) to create a layout?

You said
Bootstrap 5 Grids

The **Bootstrap 5 Grid System** is a powerful, mobile-first flexbox system used to build layouts of all shapes and sizes. It uses a **12-column system**, which means every row is divided into 12 equal parts.

1. The Grid Structure

To use the grid, you follow a strict three-step hierarchy:

1. **Container:** Wraps the entire grid (as discussed previously).
2. **Row:** Acts as a wrapper for columns (clears the container padding).
3. **Columns:** The actual content holders that sit inside rows.

Basic Example

HTML

```
<div class="container">
  <div class="row">
    <div class="col">Column 1 of 3</div>
    <div class="col">Column 2 of 3</div>
    <div class="col">Column 3 of 3</div>
  </div>
</div>
```

In this example, Bootstrap automatically divides the space into 3 equal parts.

2. Setting Column Widths

Since the grid is based on 12 units, you can specify how much space a column takes using the `.col-X` class, where **X** is a number from 1 to 12.

- `.col-4`: Takes up 1/3 of the row ($12/4=3$).
- `.col-6`: Takes up half the row ($12/6=2$).
- `.col-12`: Takes up the full width.

HTML

```
<div class="row">
  <div class="col-8">Main Content (wider)</div>
  <div class="col-4">Sidebar (narrower)</div>
</div>
```

3. Responsive Breakpoints

One of the best features of the grid is the ability to change column sizes based on the screen size. Bootstrap uses "infixes" to handle this:

Infix	Screen Size	Description
(None)	<576px	Extra Small (Phones)
sm	≥576px	Small (Large Phones)
md	≥768px	Medium (Tablets)
lg	≥992px	Large (Desktops)
xl	≥1200px	Extra Large (Large Desktops)

Export to Sheets

Responsive Example

If you want a column to be full-width on mobile but half-width on tablets, you would use:

HTML

```
<div class="col-12 col-md-6">
  This is stacked on mobile and side-by-side on tablets.
</div>
```

4. Key Grid Utilities

- **Equal-width:** Use just `.col` to let Bootstrap handle the math automatically.
- **Variable width content:** Use `.col-auto` to make the column as wide as its content.
- **Gutters:** These are the gaps between columns. You can change them using `.g-*` classes (e.g., `.g-3` for standard spacing or `.g-0` to remove them entirely).
- **Alignment:** Use flexbox utilities on the `.row` to align columns (e.g., `.align-items-center` or `.justify-content-center`).

Grid Challenge

Try creating a "Pricing Section" layout. You'll need:

1. A **Container**.
2. A **Row**.
3. Three **Columns** that are full-width on mobile (`col-12`) but take up equal space on medium screens and up (`col-md-4`).

Would you like to see a code snippet for that layout, or should we move on to **Typography** and **Tables**?

You said
Bootstrap 5 Text/Typography

In Bootstrap 5, **Typography** is designed to be clean, accessible, and responsive. It uses a "native font stack" that automatically selects the best font for your operating system (like San Francisco for iOS or Segoe UI for Windows).

1. Headings

Bootstrap styling applies to standard `<h1>` through `<h6>` tags. However, if you want to make a paragraph look like a heading without using the actual HTML tag, you can use heading classes.

- **Standard Headings:** `<h1>` to `<h6>`
- **Heading Classes:** `.h1` to `.h6` (useful for maintaining semantic structure while changing visual size).

Display Headings

When you need a heading to really stand out (like on a landing page), use **Display headings**. These are larger and more "opinionated" than standard headings.

HTML

```
<h1 class="display-1">Display 1</h1>
```

```
<h1 class="display-2">Display 2</h1>
<h1 class="display-3">Display 3</h1>
<h1 class="display-4">Display 4</h1>
<h1 class="display-5">Display 5</h1>
<h1 class="display-6">Display 6</h1>
```

2. Lead and Body Text

Standard body text has a default `font-size` and `line-height`. To make a paragraph stand out (like an introductory summary), use the `.lead` class.

HTML

```
<p class="lead"> This is a lead paragraph. It stands out from regular
text.</p>
<p> This is standard body text for comparison.</p>
```

3. Inline Text Elements

Bootstrap provides classes to style traditional HTML inline tags quickly:

- **.mark:** Highlights text (mimics `<mark>`).
 - **.small:** Decreases font size to 80% (mimics `<small>`).
 - **.text-decoration-none:** Removes underlines (great for links).
 - **.text-lowercase / .text-uppercase / .text-capitalize:** Quickly changes text casing.
-

4. Text Alignment and Wrapping

You can align text responsively using utility classes. This is extremely helpful for mobile design.

Class	Result
<code>.text-start</code>	Left aligned (LTR)
<code>.text-center</code>	Centered
<code>.text-end</code>	Right aligned (LTR)
<code>.text-nowrap</code>	Prevents text from wrapping to a new line

Export to Sheets

Responsive Alignment Example: `<p class="text-center text-md-start">Center on mobile, left-aligned on tablet/desktop.</p>`

5. Text Colors (Utilities)

Bootstrap includes several contextual classes to change the color of your text.

HTML

```
<p class="text-primary">Blue text (Primary)</p>
<p class="text-secondary">Grey text (Secondary)</p>
<p class="text-success">Green text (Success)</p>
<p class="text-danger">Red text (Danger)</p>
<p class="text-muted">Faded grey text</p>
```

6. Blockquotes

For quoting blocks of content from another source within your document, use the `.blockquote` class.

HTML

```
<figure>
  <blockquote class="blockquote">
    <p>A well-known quote, contained in a blockquote element.</p>
  </blockquote>
  <figcaption class="blockquote-footer">
    Someone famous in <cite title="Source Title">Source Title</cite>
  </figcaption>
</figure>
```

Would you like to try a **Typography challenge** where we build a hero section header, or move straight into **Bootstrap Tables**?

Sources

You said

Bootstrap 5 Colors

In Bootstrap 5, colors are handled through **utility classes** that apply to text, backgrounds, and borders. These colors are "contextual," meaning they are named based on their intent (e.g., "success" for positive actions) rather than just the color name.

1. The Core Color Palette

Bootstrap uses a standard set of colors that remain consistent across all components.

Purpose	Color	Class Suffix
Primary	Blue	-primary
Secondary	Grey	-secondary
Success	Green	-success
Danger	Red	-danger
Warning	Yellow	-warning
Info	Cyan	-info
Light	Off-white	-light
Dark	Near-black	-dark
Export to Sheets		

2. Text Colors

To change the color of your text, use the `.text-{color}` utility.

HTML

```
<p class="text-primary">This is primary blue text.</p>
<p class="text-success">This is success green text.</p>
<p class="text-danger">This is danger red text.</p>
<p class="text-muted">This is muted (grey) text.</p>
<p class="text-white bg-dark">This is white text on a dark background.</p>
```

Opacity

Bootstrap 5.1+ allows you to change the **opacity** of text using `.text-opacity-{value}` (values: 25, 50, 75). *Example:* `<p class="text-primary text-opacity-50">Faded blue text</p>`

3. Background Colors

To change the background of an element, use the `.bg-{color}` utility.

HTML

```
<div class="p-3 mb-2 bg-primary text-white">.bg-primary</div>
```

```
<div class="p-3 mb-2 bg-warning text-dark">.bg-warning</div>
```

Background Gradients

If you want a subtle vertical gradient instead of a flat color, add the `.bg-gradient` class.

HTML

```
<div class="p-3 bg-success bg-gradient text-white">Success with  
Gradient</div>
```

4. Background Subtlety (Subtle Colors)

Bootstrap 5.3 introduced **subtle** background and border colors, which are perfect for alerts or light-themed UI elements. These are much lighter than the standard palette.

- **Class:** `.bg-primary-subtle`, `.bg-danger-subtle`, etc.

[Image comparison of Bootstrap 5 standard background colors versus subtle background colors]

5. Border Colors

You can also apply these colors to borders. Note that you must first add the `.border` class to make the border visible.

HTML

```
<div class="border border-primary">Blue Border</div>  
<div class="border border-success border-3">Thick Green Border</div>
```

6. Links and Buttons

Colors also apply to interactive elements:

- **Links:** `.link-primary`, `.link-success` (these include hover states).
- **Buttons:** `.btn-primary`, `.btn-outline-danger`.

Practical Tip: Contrast

When using background colors, always ensure your text color is readable. Use `.text-white` or `.text-dark` to create high contrast.

For example, **Warning** (yellow) usually looks best with `.text-dark`, while **Primary** (blue) or **Danger** (red) almost always requires `.text-white`.

Would you like to see how to use these colors to build **Bootstrap 5 Buttons** or **Alerts**?

You said
Bootstrap 5 Tables

In Bootstrap 5, tables are designed to be clean and legible. By default, a standard HTML `<table>` has almost no styling. Adding the `.table` class instantly applies light padding and horizontal dividers.

1. Basic Table Structure

To get started, simply add the `.table` class to your `<table>` tag.

HTML

```
<table class="table">
  <thead>
    <tr>
      <th scope="col">#</th>
      <th scope="col">First</th>
      <th scope="col">Last</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">1</th>
      <td>Mark</td>
      <td>Otto</td>
    </tr>
  </tbody>
</table>
```

2. Table Variations

You can combine multiple classes to change the look and feel of your data.

Class	Effect
<code>.table-striped</code>	Adds "zebra-stripping" (alternating grey/white rows).
<code>.table-hover</code>	Highlights a row when the mouse pointer hovers over it.
<code>.table-bordered</code>	Adds borders on all sides of the table and cells.
<code>.table-borderless</code>	Removes all borders from the table.
<code>.table-sm</code>	Cuts cell padding in half to make the table more compact.

Export to Sheets

3. Table Colors

You can apply contextual colors to the entire table, specific rows, or even individual cells using the `.table-{color}` classes.

- **Dark Table:** Use `.table-dark` for a black background with white text.
- **Colored Rows:** Use `.table-primary`, `.table-success`, `.table-danger`, etc.

HTML

```
<tr class="table-success">
  <td>This row is green to indicate a successful status.</td>
</tr>
```

4. Table Head Options

Sometimes you want the header to stand out from the rest of the body. You can style the `<thead>` specifically.

- **Light Head:** `<thead class="table-light">`
 - **Dark Head:** `<thead class="table-dark">`
-

5. Responsive Tables

Tables can often break a mobile layout because they are naturally wide. To fix this, wrap your `<table>` in a `<div>` with the `.table-responsive` class.

HTML

```
<div class="table-responsive">
  <table class="table">
    ...
  </table>
</div>
```

This adds a horizontal scrollbar on small screens while keeping the rest of your page layout intact.

6. Vertical Alignment

If your cells have varying amounts of content, you can align the text vertically using:

- `.align-top`
- `.align-middle` (common for professional dashboards)
- `.align-bottom`

Would you like to try a **Table Challenge** where you create a striped, hoverable table with a dark header?

You said
Bootstrap 5 Images

In Bootstrap 5, styling images is primarily about making them **responsive** so they don't overflow their parent containers, and adding quick decorative touches like rounded corners or shadows.

1. Responsive Images

By default, images in HTML are not responsive. If an image is 1000px wide and the screen is only 400px wide, the image will stick out.

To fix this, apply the `.img-fluid` class. This sets `max-width: 100%` and `height: auto`.

HTML

```

```

2. Image Thumbnails

If you want to give an image a slight rounded 1px border appearance (common for profile pictures or gallery previews), use the `.img-thumbnail` class.

HTML

```

```

3. Aligning Images

Images are inline elements by default. To align them, you can use float utilities or text alignment on the parent container.

Using Floats

- `.float-start`: Aligns the image to the left.
- `.float-end`: Aligns the image to the right.

Using Centering

To center an image, you must make it a block element using `.d-block` and then use horizontal auto-margins with `.mx-auto`.

HTML

```

```

4. Rounded Corners

Since Bootstrap 5 provides utility classes for borders, you can easily change the shape of your images without writing custom CSS.

Class	Effect
<code>.rounded</code>	Small rounded corners on all sides.
<code>.rounded-circle</code>	Turns the image into a perfect circle (best for square images).
<code>.rounded-pill</code>	Creates a pill-like shape.
<code>.rounded-top</code>	Only rounds the top two corners.

Export to Sheets

5. The `<picture>` Element

If you are using the HTML5 `<picture>` element to serve different images for different screen sizes, make sure to apply the `.img-fluid` class to the `<img>` tag inside the picture element, not the `<picture>` tag itself.

HTML

```
<picture>
  <source srcset="..." type="image/svg+xml">
  
</picture>
```

6. Figures

If you need to display an image with a caption, Bootstrap uses the `<figure>` and `<figcaption>` elements paired with specific classes.

HTML

```
<figure class="figure">
  
  <figcaption class="figure-caption text-end">A beautiful morning in the
  forest.</figcaption>
</figure>
```

Would you like to see how to combine these images into a **Bootstrap 5 Card** component, which is the most common way to display "Profile" or "Product" info?

You said
Bootstrap 5 Jumbotron

In **Bootstrap 5**, the specific `.jumbotron` class from version 4 has been **removed**.

Instead of a single dedicated class, Bootstrap 5 encourages you to use a combination of **Utility Classes** (like spacing, colors, and borders) to create a "Jumbotron" effect. This gives you more flexibility to customize the look.

1. Creating a Basic Jumbotron

To recreate the classic Jumbotron look, you typically use a `<div>` with background color, padding, and rounded corners.

HTML

```
<div class="p-5 mb-4 bg-light rounded-3">
  <div class="container-fluid py-5">
    <h1 class="display-5 fw-bold">Custom Jumbotron</h1>
    <p class="col-md-8 fs-4">Using a series of utilities, you can create this
jumbotron, just like in previous versions of Bootstrap.</p>
    <button class="btn btn-primary btn-lg" type="button">Example
button</button>
  </div>
</div>
```

2. Key Classes Used

When building your own Jumbotron, these are the "building blocks" you'll use:

Class	Purpose
<code>.p-5</code>	Adds large padding inside the box.
<code>.mb-4</code>	Adds a margin at the bottom to separate it from content.
<code>.bg-light</code>	Sets a light grey background (or use <code>.bg-dark</code> , <code>.bg-primary</code>).
<code>.rounded-3</code>	Adds rounded corners to the box.
<code>.display-5</code>	Makes the heading large and prominent.
<code>.fw-bold</code>	Makes the text bold.
<code>.fs-4</code>	Sets a specific font size for the paragraph.

Export to Sheets

3. Dark Mode Jumbotron

You can easily switch the theme by swapping the background and text color utilities.

HTML

```
<div class="p-5 text-white bg-dark rounded-3">
  <h2>Dark Mode Hero</h2>
  <p>This version uses dark background utilities for a high-contrast
look.</p>
  <hr class="my-4">
  <p>It uses utility classes for typography and spacing to match your
brand.</p>
```

```
<button class="btn btn-outline-light" type="button">Learn More</button>
</div>
```

4. Full-Width (Fluid) Jumbotron

If you want the Jumbotron to span the entire width of the screen without rounded corners, remove the `.container` or `.rounded-3` classes.

HTML

```
<div class="p-5 bg-primary text-white">
  <div class="container">
    <h1 class="display-4">Full Width Hero</h1>
    <p class="lead">This spreads across the entire viewport.</p>
  </div>
</div>
```

Summary: Why did it change?

Bootstrap 4's `.jumbotron` was a rigid component. By moving to **Utility Classes** in Version 5:

- **Less CSS:** The framework is lighter because it doesn't need a specific "Jumbotron" style definition.
- **Total Control:** You can decide exactly how much padding, what color, and how rounded you want the box to be without fighting default styles.

Would you like to see how to add a **Background Image** to your Jumbotron using a little bit of custom CSS?

Sources

You said

Bootstrap 5 Alerts

Alerts in Bootstrap 5 are used to provide contextual feedback messages for typical user actions (like submitting a form) or system statuses (like an error or a success message).

1. Basic Alert Structure

To create an alert, use a `<div>` with the `.alert` base class and a contextual color class like `.alert-primary`, `.alert-success`, etc.

HTML

```
<div class="alert alert-primary" role="alert">
  A simple primary alert—check it out!
</div>
```

2. Contextual Variations

Bootstrap provides several colors to convey different meanings:

Class	Purpose	Color
<code>.alert-success</code>	Positive action or state	Green
<code>.alert-danger</code>	Error or dangerous action	Red
<code>.alert-warning</code>	Warning or attention needed	Yellow
<code>.alert-info</code>	General informative message	Blue/Cyan
<code>.alert-light</code>	Subtle, light background	Light Grey
<code>.alert-dark</code>	High contrast, dark background	Dark Grey

Export to Sheets

3. Alerts with Links

If you put a link inside an alert, use the `.alert-link` utility class. This ensures the link color matches the alert theme and remains readable.

HTML

```
<div class="alert alert-info" role="alert">
  A simple info alert with <a href="#" class="alert-link">an example
  link</a>.
</div>
```

4. Additional Content

Alerts can contain more than just a single line of text. You can include headings, paragraphs, and dividers.

HTML

```
<div class="alert alert-success" role="alert">
  <h4 class="alert-heading">Well done!</h4>
```

```
<p>Aww yeah, you successfully read this important alert message.</p>
<hr>
<p class="mb-0">Whenever you need to, be sure to use margin utilities to
keep things tidy.</p>
</div>
```

5. Dismissible Alerts (Close Button)

You can allow users to dismiss an alert by clicking a "close" button. This requires the `.alert-dismissible` class and the Bootstrap JavaScript bundle.

HTML

```
<div class="alert alert-warning alert-dismissible fade show" role="alert">
  <strong>Holy guacamole!</strong> You should check in on some of those
  fields below.
  <button type="button" class="btn-close" data-bs-dismiss="alert" aria-
  label="Close"></button>
</div>
```

- **.fade.show:** Adds a smooth fading animation when the alert is closed.
 - **.btn-close:** The specific Bootstrap class for the "X" icon.
 - **data-bs-dismiss="alert":** The JavaScript trigger that makes the button work.
-

6. Icons in Alerts

Bootstrap doesn't include icons by default, but you can easily add **Bootstrap Icons** (or FontAwesome) to make your alerts more visual.

HTML

```
<div class="alert alert-primary d-flex align-items-center" role="alert">
  <i class="bi bi-info-circle-fill me-2"></i>
  <div>
    An example alert with an icon
  </div>
</div>
```

Bootstrap 5 Buttons

In Bootstrap 5, **Buttons** are highly versatile components used for actions in forms, dialogs, and more. They come with a base class and several contextual modifiers to change their appearance.

1. Basic Button Classes

To create a Bootstrap button, you **must** use the `.btn` base class. Without it, the contextual classes (like `.btn-primary`) won't apply the correct padding or alignment.

HTML

```
<button type="button" class="btn btn-primary">Primary</button>
<button type="button" class="btn btn-secondary">Secondary</button>
<button type="button" class="btn btn-success">Success</button>
<button type="button" class="btn btn-danger">Danger</button>
<button type="button" class="btn btn-warning">Warning</button>
<button type="button" class="btn btn-info">Info</button>
```

2. Button Tags

Bootstrap classes are designed to work on multiple HTML elements, including `<button>`, `<a>`, and `<input>`.

- **Link as a button:** `<a class="btn btn-primary" href="#" role="button">Link</a>`
 - **Submit button:** `<input class="btn btn-primary" type="submit" value="Submit">`
-

3. Outline Buttons

If you prefer a cleaner look without a heavy background color, use the `.btn-outline-*` classes. The background only appears when the user hovers over the button.

HTML

```
<button type="button" class="btn btn-outline-primary">Primary
Outline</button>
<button type="button" class="btn btn-outline-secondary">Secondary
Outline</button>
```

4. Button Sizes

You can easily adjust the size of your buttons using `.btn-lg` or `.btn-sm`.

Class	Description
<code>.btn-lg</code>	Large button for primary calls to action.
(Default)	Standard size.
<code>.btn-sm</code>	Small button for secondary or compact UI.

Export to Sheets

5. Block Level Buttons (Full Width)

In Bootstrap 5, "block buttons" are created using **Grid** or **Utility** classes on a parent container, rather than a specific button class.

HTML

```
<div class="d-grid gap-2">
  <button class="btn btn-primary" type="button">Button 1</button>
  <button class="btn btn-primary" type="button">Button 2</button>
</div>
```

The `.d-grid` utility makes the buttons span the full width of their parent.

6. Button States

- **Active State:** Buttons will appear pressed when clicked. You can force this state by adding the `.active` class.
- **Disabled State:** Add the `disabled` attribute to a `<button>` to make it unclickable and faded.

HTML

```
<button type="button" class="btn btn-primary" disabled>Disabled
Button</button>
```

7. Button Groups

To wrap a series of buttons together on a single line, use the `.btn-group` class.

HTML

```
<div class="btn-group" role="group" aria-label="Basic example">
  <button type="button" class="btn btn-primary">Left</button>
  <button type="button" class="btn btn-primary">Middle</button>
  <button type="button" class="btn btn-primary">Right</button>
</div>
```

Bootstrap 5 Button Groups

Button Groups in Bootstrap 5 allow you to stack a series of buttons together on a single line. They are perfect for toolbars, pagination, or switching between different views (like a "Day/Week/Month" filter).

1. Basic Button Group

To create a group, wrap a series of buttons in a `div` with the `.btn-group` class. Bootstrap will automatically remove the rounded corners between the buttons to make them look like a single unit.

HTML

```
<div class="btn-group" role="group" aria-label="Basic example">
  <button type="button" class="btn btn-primary">Left</button>
  <button type="button" class="btn btn-primary">Middle</button>
  <button type="button" class="btn btn-primary">Right</button>
</div>
```

2. Button Toolbars

If you have multiple button groups and want to space them out or combine them into a more complex layout, use the `.btn-toolbar` class. This is essentially a "container" for button groups.

HTML

```
<div class="btn-toolbar" role="toolbar" aria-label="Toolbar with button
groups">
  <div class="btn-group me-2" role="group">
    <button type="button" class="btn btn-secondary">1</button>
    <button type="button" class="btn btn-secondary">2</button>
  </div>
  <div class="btn-group" role="group">
    <button type="button" class="btn btn-secondary">3</button>
  </div>
</div>
```

3. Sizing

Instead of applying sizing classes to every individual button, you can apply them once to the `.btn-group` container.

- **Large:** `.btn-group-lg`
- **Small:** `.btn-group-sm`

[Image comparison of Bootstrap 5 button group sizes from small to large]

4. Vertical Variation

If you want the buttons to stack on top of each other rather than sitting side-by-side, swap `.btn-group` for `.btn-group-vertical`.

HTML

```
<div class="btn-group-vertical" role="group">
  <button type="button" class="btn btn-dark">Top</button>
  <button type="button" class="btn btn-dark">Middle</button>
  <button type="button" class="btn btn-dark">Bottom</button>
</div>
```

5. Nesting & Dropdowns

You can nest a button group inside another one to create dropdown menus within the group. This is common for "Split Buttons" where the main button does one thing and the arrow opens more options.

HTML

```
<div class="btn-group" role="group">
  <button type="button" class="btn btn-primary">Action</button>
  <div class="btn-group" role="group">
    <button type="button" class="btn btn-primary dropdown-toggle" data-bs-
toggle="dropdown"></button>
    <ul class="dropdown-menu">
      <li><a class="dropdown-item" href="#">Dropdown link</a></li>
    </ul>
  </div>
</div>
```

6. Checkbox and Radio Button Groups

You can turn standard checkboxes and radios into button-style toggles. This is great for settings or filters.

- **Radio Group:** Only one can be selected.
- **Checkbox Group:** Multiple can be selected.

HTML

```
<div class="btn-group" role="group">
  <input type="radio" class="btn-check" name="btnradio" id="btnradio1"
checked>
  <label class="btn btn-outline-primary" for="btnradio1">Radio 1</label>

  <input type="radio" class="btn-check" name="btnradio" id="btnradio2">
  <label class="btn btn-outline-primary" for="btnradio2">Radio 2</label>
</div>
```

Quick Accessibility Note

Always include `role="group"` and an `aria-label` on your container. Since the buttons are visually grouped, screen readers need this extra context to let users know the buttons are related.

Bootstrap 5 Badges

Badges in Bootstrap 5 are small, versatile components used to add extra information to content—like a count in a notification center, a "New" label on a product, or a status indicator.

1. Basic Badge Structure

Badges scale to match the size of the immediate parent element by using relative font sizing and `em` units. To create one, use the `.badge` base class followed by a contextual color class.

HTML

```
<h1>Example heading <span class="badge bg-secondary">New</span></h1>
<h2>Example heading <span class="badge bg-secondary">New</span></h2>
```

2. Contextual Variations

Just like buttons and alerts, badges use the standard Bootstrap color palette to convey meaning.

Class	Purpose	Color
<code>.bg-primary</code>	Main highlight	Blue
<code>.bg-success</code>	Positive/Active	Green

Class	Purpose	Color
<code>.bg-danger</code>	Error/Critical	Red
<code>.bg-warning</code>	Warning/Pending	Yellow
<code>.bg-info</code>	Information	Cyan

Export to Sheets

HTML

```
<span class="badge bg-primary">Primary</span>
<span class="badge bg-success">Success</span>
<span class="badge bg-danger">Danger</span>
```

3. Pill Badges

If you prefer a more rounded, "capsule" look, add the `.rounded-pill` utility class. This is very common for tags or category labels.

HTML

```
<span class="badge rounded-pill bg-info text-dark">Category</span>
<span class="badge rounded-pill bg-light text-dark">Draft</span>
```

4. Badges inside Buttons

One of the most common use cases for badges is showing a count inside a button, such as "Messages" or "Notifications."

HTML

```
<button type="button" class="btn btn-primary">
  Notifications <span class="badge bg-light text-dark">4</span>
</button>
```

5. Positioned (Floating) Badges

Bootstrap 5 makes it easy to "float" a badge in the corner of a button or a link. This is perfect for unread message counts. Use the utility classes for positioning (`top-0`, `start-100`, etc.).

HTML

```
<button type="button" class="btn btn-primary position-relative">
  Inbox
  <span class="position-absolute top-0 start-100 translate-middle badge
rounded-pill bg-danger">
    99+
    <span class="visually-hidden">unread messages</span>
```

```
</span>  
</button>
```

- **.position-relative:** Added to the parent so the badge stays attached to it.
 - **.translate-middle:** Centers the badge exactly on the corner.
 - **.visually-hidden:** Keeps the component accessible for screen readers.
-

6. Subtle Badges (New in v5.3)

If the standard solid colors are too "loud," you can use the subtle background classes introduced in the latest versions.

HTML

```
<span class="badge bg-primary-subtle text-primary-emphasis">Subtle  
Blue</span>  
<span class="badge bg-success-subtle text-success-emphasis">Subtle  
Green</span>
```

Bootstrap 5 Progress Bars

Progress Bars in Bootstrap 5 are used to show the status of a process, like a file upload, a setup wizard, or a loading state. They are built with two HTML elements: a container and the bar itself.

1. Basic Progress Bar

To create a progress bar, use a wrapper with the `.progress` class and an inner `div` with the `.progress-bar` class. You control the width of the bar using inline CSS or utility classes.

HTML

```
<div class="progress">  
  <div class="progress-bar" role="progressbar" style="width: 25%" aria-  
    valuenow="25" aria-valuemin="0" aria-valuemax="100"></div>  
</div>
```

2. Labels

If you want to show the percentage or a status message inside the bar, simply place the text inside the `.progress-bar` `div`.

HTML

```
<div class="progress">
  <div class="progress-bar" style="width: 75%;">75% Complete</div>
</div>
```

3. Height

By default, the progress bar is **16px** high. To change the height, you must set it on the parent `.progress` element. The inner bar will automatically adjust.

HTML

```
<div class="progress" style="height: 1px;">
  <div class="progress-bar" style="width: 50%;"></div>
</div>

<div class="progress" style="height: 30px;">
  <div class="progress-bar" style="width: 50%;"></div>
</div>
```

4. Background Colors

You can change the color of the progress bar using the standard contextual background utilities (`.bg-*`).

HTML

```
<div class="progress mb-2">
  <div class="progress-bar bg-success" style="width: 40%">Success</div>
</div>
<div class="progress mb-2">
  <div class="progress-bar bg-info" style="width: 60%">Info</div>
</div>
<div class="progress">
  <div class="progress-bar bg-danger" style="width: 90%">Danger</div>
</div>
```

5. Striped and Animated Bars

If you want to add some visual flair, you can add stripes or even animate them to show that a process is "working."

- **Striped:** Add `.progress-bar-striped`.
- **Animated:** Add `.progress-bar-animated` (this requires the stripes class to be visible).

HTML

```
<div class="progress">
  <div class="progress-bar progress-bar-striped progress-bar-animated"
style="width: 75%"></div>
</div>
```

6. Multiple Bars (Stacked)

You can include multiple progress bars inside a single `.progress` container to show a breakdown of different segments.

HTML

```
<div class="progress">
  <div class="progress-bar bg-primary" style="width: 15%">Space Used</div>
  <div class="progress-bar bg-success" style="width: 30%">Free Space</div>
  <div class="progress-bar bg-info" style="width: 20%">System</div>
</div>
```

7. Accessibility

To ensure everyone can understand the progress, always include these attributes:

- `role="progressbar"`: Defines the element as a progress bar.
- `aria-valuenow`: The current numerical value.
- `aria-valuemin`: The minimum value (usually 0).
- `aria-valuemax`: The maximum value (usually 100).

Bootstrap 5 Spinners

Spinners in Bootstrap 5 are used to show the loading state of a component or page. They are built entirely with HTML and CSS, meaning you don't need any JavaScript to make them spin.

1. Border Spinner

The default spinner is a rotating border. Use the `.spinner-border` class to create one.

HTML

```
<div class="spinner-border" role="status">
  <span class="visually-hidden">Loading...</span>
</div>
```

- **.visually-hidden:** This is important for accessibility. It ensures screen readers announce "Loading..." while sighted users see the animation.
-

2. Growing Spinner

If you prefer a "pulsing" effect instead of a spinning one, use the `.spinner-grow` class.

HTML

```
<div class="spinner-grow" role="status">
  <span class="visually-hidden">Loading...</span>
</div>
```

3. Colors

You can use any of the standard Bootstrap **text color utilities** to change the color of a spinner.

HTML

```
<div class="spinner-border text-primary" role="status"></div>
<div class="spinner-border text-success" role="status"></div>
<div class="spinner-border text-danger" role="status"></div>
<div class="spinner-border text-warning" role="status"></div>
```

4. Size

Spinners are sized based on `em` units, but Bootstrap provides two specific utility classes for quick scaling:

Class	Description
<code>.spinner-border-sm</code>	A small version (usually for buttons).
<code>.spinner-grow-sm</code>	A small version of the growing spinner.
Custom Style	Use <code>style="width: 3rem; height: 3rem;"</code> for custom sizes.
Export to Sheets	

5. Buttons with Spinners

This is the most common way to use spinners. It lets the user know that their click was registered and the system is processing.

HTML

```
<button class="btn btn-primary" type="button" disabled>
  <span class="spinner-border spinner-border-sm" role="status" aria-
hidden="true"></span>
  Loading...
</button>

<button class="btn btn-primary" type="button" disabled>
  <span class="spinner-grow spinner-grow-sm" role="status" aria-
hidden="true"></span>
  Processing...
</button>
```

6. Alignment

Spinners are `inline-block` elements, so you can align them using text utilities or flexbox.

- **Margin:** Use `.m-5` for a simple margin.
- **Flexbox:** Wrap it in a `d-flex` container.

HTML

```
<div class="d-flex justify-content-center">
  <div class="spinner-border" role="status"></div>
</div>
```

Bootstrap 5 Pagination

Pagination in Bootstrap 5 is used to let users navigate through large amounts of content that has been split into multiple pages (like search results or blog archives).

1. Basic Pagination

To create pagination, wrap a series of `<li>` items in a `<ul>` with the `.pagination` class. Each link inside must have the `.page-link` class, and its parent `<li>` must have the `.page-item` class.

HTML

```
<nav aria-label="Page navigation example">
  <ul class="pagination">
    <li class="page-item"><a class="page-link" href="#">Previous</a></li>
    <li class="page-item"><a class="page-link" href="#">1</a></li>
    <li class="page-item"><a class="page-link" href="#">2</a></li>
    <li class="page-item"><a class="page-link" href="#">3</a></li>
    <li class="page-item"><a class="page-link" href="#">Next</a></li>
```

```
</ul>
</nav>
```

2. Active and Disabled States

You can indicate the current page or a link that cannot be clicked (like "Previous" on the first page).

- **Active:** Add `.active` to the `.page-item`.
- **Disabled:** Add `.disabled` to the `.page-item`.

HTML

```
<ul class="pagination">
  <li class="page-item disabled"><a class="page-link">Previous</a></li>
  <li class="page-item active" aria-current="page"><a class="page-link"
href="#">1</a></li>
  <li class="page-item"><a class="page-link" href="#">2</a></li>
</ul>
```

3. Sizing

Need larger or smaller navigation? Apply sizing classes directly to the `<ul>` element.

Class	Description
<code>.pagination-lg</code>	Large pagination for high visibility.
(Default)	Standard size.
<code>.pagination-sm</code>	Small pagination for compact layouts.
Export to Sheets	

[Image comparison of Bootstrap 5 pagination sizes from small to large]

4. Alignment

Since pagination is built with flexbox, you can easily change its horizontal alignment using Bootstrap's flex utilities on the `<ul>` tag.

- **Center:** `<ul class="pagination justify-content-center">`
 - **Right:** `<ul class="pagination justify-content-end">`
-

5. Using Icons

Instead of text like "Previous" or "Next," you can use HTML entities or icons. Be sure to include a `visually-hidden` span for screen readers.

HTML

```
<li class="page-item">
  <a class="page-link" href="#" aria-label="Next">
    <span aria-hidden="true">&raquo;</span>
    <span class="visually-hidden">Next</span>
  </a>
</li>
```

6. Accessibility Tips

- **Use `<nav>`:** Always wrap your pagination in a `<nav>` element to identify it as a navigation section.
- **Label it:** Use `aria-label="Page navigation"` on the `<nav>` so screen readers can distinguish it from other navigation on the page.
- **Current Page:** Use `aria-current="page"` on the active `.page-item` to let users know exactly where they are.

Bootstrap 5 List Groups

List Groups in Bootstrap 5 are flexible and powerful components for displaying series of content. They can be used for simple lists, complex custom content, or even as navigation.

1. Basic List Group

The most basic list group is an unordered list with `.list-group` and list items with `.list-group-item`.

HTML

```
<ul class="list-group">
  <li class="list-group-item">An item</li>
  <li class="list-group-item">A second item</li>
  <li class="list-group-item">A third item</li>
</ul>
```

2. Active and Disabled Items

You can highlight the current selection or gray out an item that isn't available.

- **Active:** Add `.active` to a `.list-group-item`.
 - **Disabled:** Add `.disabled` to a `.list-group-item`.
-

3. Flush List Group

If you want to remove the outer borders and rounded corners (to make the list fit edge-to-edge inside a parent container like a Card), use the `.list-group-flush` class.

HTML

```
<ul class="list-group list-group-flush">
  <li class="list-group-item">First item</li>
  <li class="list-group-item">Second item</li>
</ul>
```

4. Linked Items (Actionable)

By swapping `<ul>` and `<li>` for `<div>` and `<a>`, you can create hoverable, clickable list items. Use the `.list-group-item-action` class to add a hover effect.

HTML

```
<div class="list-group">
  <a href="#" class="list-group-item list-group-item-action active">The
current link</a>
  <a href="#" class="list-group-item list-group-item-action">A second
link</a>
  <a href="#" class="list-group-item list-group-item-action disabled">A
disabled link</a>
</div>
```

5. Contextual Classes

Use contextual classes to style list items with specific colors to convey meaning (e.g., success, danger).

- `.list-group-item-primary`
- `.list-group-item-success`
- `.list-group-item-danger`

6. With Badges and Checkboxes

List groups work perfectly with other Bootstrap components. You can use flexbox utilities (`d-flex`, `justify-content-between`) to align badges or checkboxes.

HTML

```
<ul class="list-group">
  <li class="list-group-item d-flex justify-content-between align-items-center">
    A list item
    <span class="badge bg-primary rounded-pill">14</span>
  </li>
</ul>
```

7. Horizontal List Groups

If you want the items to sit side-by-side instead of stacking vertically, use the `.list-group-horizontal` class. You can also make it responsive (e.g., `.list-group-horizontal-md`).

HTML

```
<ul class="list-group list-group-horizontal">
  <li class="list-group-item">Item 1</li>
  <li class="list-group-item">Item 2</li>
  <li class="list-group-item">Item 3</li>
</ul>
```

8. Custom Content

List groups can contain almost any HTML. Use flexbox to create complex layouts like a "Message Inbox" style.

HTML

```
<div class="list-group">
  <a href="#" class="list-group-item list-group-item-action">
    <div class="d-flex w-100 justify-content-between">
      <h5 class="mb-1">List group item heading</h5>
      <small>3 days ago</small>
    </div>
    <p class="mb-1">Some placeholder content in a paragraph.</p>
    <small>And some muted small print.</small>
  </a>
</div>
```

Bootstrap 5 Cards

Cards are the most popular and flexible component in Bootstrap 5. They provide a bordered box with a small amount of padding around its content, making them perfect for displaying "objects" like products, user profiles, or blog posts.

1. Basic Card Structure

A standard card is built with a wrapper (`.card`) and a body (`.card-body`). It's recommended to set a width on the card (either with a fixed style or grid columns) so it doesn't span the full width of the page.

HTML

```
<div class="card" style="width: 18rem;">
  <div class="card-body">
    <h5 class="card-title">Card Title</h5>
    <h6 class="card-subtitle mb-2 text-muted">Card subtitle</h6>
    <p class="card-text">Some quick example text to build on the card
title.</p>
    <a href="#" class="card-link">Card link</a>
  </div>
</div>
```

2. Content Types

Cards are highly modular. You can mix and match different "sections" to create the layout you need.

Component	Class	Description
Header	<code>.card-header</code>	A separate bar at the top of the card.
Footer	<code>.card-footer</code>	A separate bar at the bottom.
Image	<code>.card-img-top</code>	Places an image flush at the top.
Image Overlay	<code>.card-img-overlay</code>	Turns the image into a background with text on top.

Export to Sheets

Example with Image and Button

HTML

```
<div class="card" style="width: 18rem;">
  
  <div class="card-body">
    <h5 class="card-title">Nature Walk</h5>
    <p class="card-text">Explore the beauty of the woods.</p>
```

```
    <a href="#" class="btn btn-primary">Go somewhere</a>
  </div>
</div>
```

3. List Groups inside Cards

You can embed a **List Group** inside a card to create a structured "sidebar" or "specifications" box. Use `.list-group-flush` to make the list edge-to-edge.

HTML

```
<div class="card" style="width: 18rem;">
  <div class="card-header">Featured</div>
  <ul class="list-group list-group-flush">
    <li class="list-group-item">An item</li>
    <li class="list-group-item">A second item</li>
  </ul>
</div>
```

4. Alignment and Navigation

- **Alignment:** Use text utilities like `.text-center` or `.text-end` on the `.card` to align all content inside.
- **Navigation:** You can even put **Tabs** or **Pills** in the card header to create a tabbed interface.

HTML

```
<div class="card text-center">
  <div class="card-header">
    <ul class="nav nav-tabs card-header-tabs">
      <li class="nav-item"><a class="nav-link active"
href="#">Active</a></li>
      <li class="nav-item"><a class="nav-link" href="#">Link</a></li>
    </ul>
  </div>
  <div class="card-body">
    <p class="card-text">Content changes based on tabs.</p>
  </div>
</div>
```

5. Horizontal Cards

Sometimes a vertical stack doesn't fit your design. You can use the **Grid System** inside a card to place an image on the left and text on the right.

HTML

```
<div class="card mb-3" style="max-width: 540px;">
  <div class="row g-0">
    <div class="col-md-4">
      
    </div>
    <div class="col-md-8">
      <div class="card-body">
        <h5 class="card-title">Card title</h5>
        <p class="card-text">This is a wider card with text on the right.</p>
      </div>
    </div>
  </div>
</div>
```

6. Card Groups and Grids

If you want to display multiple cards together:

- **Card Group:** (`.card-group`) Attaches cards together as a single unit with equal heights.
- **Grid:** Use standard `.row-cols-*` classes to create a responsive grid of cards with gaps.

HTML

```
<div class="row row-cols-1 row-cols-md-3 g-4">
  <div class="col">
    <div class="card">...</div>
  </div>
  <div class="col">
    <div class="card">...</div>
  </div>
</div>
```

Bootstrap 5 Cards

Cards are the most popular and flexible component in Bootstrap 5. They provide a bordered box with a small amount of padding around its content, making them perfect for displaying "objects" like products, user profiles, or blog posts.

1. Basic Card Structure

A standard card is built with a wrapper (`.card`) and a body (`.card-body`). It's recommended to set a width on the card (either with a fixed style or grid columns) so it doesn't span the full width of the page.

HTML

```
<div class="card" style="width: 18rem;">
  <div class="card-body">
    <h5 class="card-title">Card Title</h5>
    <h6 class="card-subtitle mb-2 text-muted">Card subtitle</h6>
    <p class="card-text">Some quick example text to build on the card
title.</p>
    <a href="#" class="card-link">Card link</a>
  </div>
</div>
```

2. Content Types

Cards are highly modular. You can mix and match different "sections" to create the layout you need.

Component	Class	Description
Header	<code>.card-header</code>	A separate bar at the top of the card.
Footer	<code>.card-footer</code>	A separate bar at the bottom.
Image	<code>.card-img-top</code>	Places an image flush at the top.
Image Overlay	<code>.card-img-overlay</code>	Turns the image into a background with text on top.

Export to Sheets

Example with Image and Button

HTML

```
<div class="card" style="width: 18rem;">
  
  <div class="card-body">
    <h5 class="card-title">Nature Walk</h5>
    <p class="card-text">Explore the beauty of the woods.</p>
    <a href="#" class="btn btn-primary">Go somewhere</a>
  </div>
</div>
```

3. List Groups inside Cards

You can embed a **List Group** inside a card to create a structured "sidebar" or "specifications" box. Use `.list-group-flush` to make the list edge-to-edge.

HTML

```
<div class="card" style="width: 18rem;">
  <div class="card-header">Featured</div>
  <ul class="list-group list-group-flush">
    <li class="list-group-item">An item</li>
```

```
    <li class="list-group-item">A second item</li>
  </ul>
</div>
```

4. Alignment and Navigation

- **Alignment:** Use text utilities like `.text-center` or `.text-end` on the `.card` to align all content inside.
- **Navigation:** You can even put **Tabs** or **Pills** in the card header to create a tabbed interface.

HTML

```
<div class="card text-center">
  <div class="card-header">
    <ul class="nav nav-tabs card-header-tabs">
      <li class="nav-item"><a class="nav-link active"
href="#">Active</a></li>
      <li class="nav-item"><a class="nav-link" href="#">Link</a></li>
    </ul>
  </div>
  <div class="card-body">
    <p class="card-text">Content changes based on tabs.</p>
  </div>
</div>
```

5. Horizontal Cards

Sometimes a vertical stack doesn't fit your design. You can use the **Grid System** inside a card to place an image on the left and text on the right.

HTML

```
<div class="card mb-3" style="max-width: 540px;">
  <div class="row g-0">
    <div class="col-md-4">
      
    </div>
    <div class="col-md-8">
      <div class="card-body">
        <h5 class="card-title">Card title</h5>
        <p class="card-text">This is a wider card with text on the right.</p>
      </div>
    </div>
  </div>
</div>
```

6. Card Groups and Grids

If you want to display multiple cards together:

- **Card Group:** (`.card-group`) Attaches cards together as a single unit with equal heights.
- **Grid:** Use standard `.row-cols-*` classes to create a responsive grid of cards with gaps.

HTML

```
<div class="row row-cols-1 row-cols-md-3 g-4">
  <div class="col">
    <div class="card">...</div>
  </div>
  <div class="col">
    <div class="card">...</div>
  </div>
</div>
```

Bootstrap 5 Collapse

The **Collapse** component in Bootstrap 5 is used to toggle the visibility of content. It's the foundation for more complex components like **Accordions**. It uses a smooth CSS transition to slide content up or down.

1. Basic Example

To make an element collapsible, you need a **trigger** (like a button or a link) and the **target content**.

- **The Trigger:** Needs `data-bs-toggle="collapse"` and `data-bs-target="#id"`.
- **The Target:** Needs the `.collapse` class and a unique `id` that matches the trigger's target.

HTML

```
<p>
  <button class="btn btn-primary" type="button" data-bs-toggle="collapse"
data-bs-target="#collapseExample">
    Toggle Content
  </button>
</p>

<div class="collapse" id="collapseExample">
  <div class="card card-body">
    Some placeholder content for the collapse component. This panel is hidden
by default but revealed when the user activates the relevant trigger.
  </div>
</div>
```

```
</div>
</div>
```

2. Horizontal Collapse

By default, the collapse component animates the `height`. If you want it to animate the `width` instead, add the `.collapse-horizontal` class to the collapsing element and set a fixed `width` on its immediate child.

HTML

```
<div class="collapse collapse-horizontal" id="collapseWidthExample">
  <div class="card card-body" style="width: 300px;">
    This content collapses horizontally instead of vertically.
  </div>
</div>
```

3. Multiple Targets

A single button can toggle multiple elements, or multiple buttons can toggle the same element.

- To toggle multiple elements with one button, use a class selector in the `data-bs-target` (e.g., `data-bs-target=".multi-collapse"`).
-

4. Accordion (Grouped Collapse)

The **Accordion** is a special implementation of the collapse component where only one item stays open at a time. It uses the `.accordion` wrapper and requires a `data-bs-parent` attribute on the collapsible items to coordinate their states.

HTML

```
<div class="accordion" id="accordionExample">
  <div class="accordion-item">
    <h2 class="accordion-header">
      <button class="accordion-button" type="button" data-bs-
toggle="collapse" data-bs-target="#collapseOne">
        Accordion Item #1
      </button>
    </h2>
    <div id="collapseOne" class="accordion-collapse collapse show" data-bs-
parent="#accordionExample">
      <div class="accordion-body">
```

```
        <strong>This is the first item's accordion body.</strong> It is shown  
by default.  
    </div>  
  </div>  
</div>  
</div>
```

5. Helpful Classes & Attributes

Class/Attribute	Purpose
<code>.collapse</code>	Hides the content (default state).
<code>.collapse.show</code>	Shows the content (expanded state).
<code>.collapsing</code>	Applied during the transition (do not apply manually).
<code>data-bs-toggle="collapse"</code>	Tells Bootstrap's JS to handle the click.
<code>aria-expanded="true/false"</code>	Crucial for Accessibility. Tells screen readers if the content is open.

Export to Sheets

Pro-Tip: JavaScript Methods

If you want to trigger a collapse via code (for example, after a timer or a specific user action), you can use the Bootstrap Constructor:

JavaScript

```
const myCollapse = document.getElementById('myCollapse');  
const bsCollapse = new bootstrap.Collapse(myCollapse, {  
  toggle: false  
});  
  
// To show it:  
bsCollapse.show();
```

Bootstrap 5 Navs

Navs in Bootstrap 5 are flexible components used to build navigation headers, sidebars, and tabbed interfaces. They are built using a base `.nav` class and can be customized with various modifiers.

1. Basic Nav Structure

A standard nav is created using a `<ul>` with the `.nav` class and `<li>` items with `.nav-item`. The links inside use `.nav-link`.

HTML

```
<ul class="nav">
  <li class="nav-item">
    <a class="nav-link active" aria-current="page" href="#">Active</a>
  </li>
  <li class="nav-item">
    <a class="nav-link" href="#">Link</a>
  </li>
  <li class="nav-item">
    <a class="nav-link disabled">Disabled</a>
  </li>
</ul>
```

2. Alignment Options

Since navs use flexbox, you can easily change the horizontal alignment using utility classes on the `<ul>` element.

- **Center:** `.justify-content-center`
 - **Right:** `.justify-content-end`
 - **Vertical:** `.flex-column` (Great for sidebars)
-

3. Visual Styles (Tabs vs. Pills)

Bootstrap provides two primary "skins" for navigation to make them look like interactive UI elements.

Nav Tabs

The `.nav-tabs` class creates a tabbed interface with borders. This is usually paired with JavaScript to switch content.

HTML

```
<ul class="nav nav-tabs">
  <li class="nav-item">
    <a class="nav-link active" href="#">Home</a>
  </li>
  <li class="nav-item">
    <a class="nav-link" href="#">Profile</a>
  </li>
</ul>
```

Nav Pills

The `.nav-pills` class creates rounded "pill" buttons for each item.

HTML

```
<ul class="nav nav-pills">
  <li class="nav-item">
    <a class="nav-link active" href="#">Home</a>
  </li>
  <li class="nav-item">
    <a class="nav-link" href="#">Profile</a>
  </li>
</ul>
```

4. Fill and Justify

If you want your navigation items to take up all the available width, use these classes:

- **.nav-fill:** Proportionally fills the space based on the content of each item.
 - **.nav-justified:** Makes every nav item exactly the same width.
-

5. Working with Dropdowns

You can easily add a dropdown menu to a nav item by combining the Nav component with the **Dropdown** component.

HTML

```
<ul class="nav nav-pills">
  <li class="nav-item">
    <a class="nav-link active" href="#">Active</a>
  </li>
  <li class="nav-item dropdown">
    <a class="nav-link dropdown-toggle" data-bs-toggle="dropdown" href="#"
role="button">Dropdown</a>
    <ul class="dropdown-menu">
      <li><a class="dropdown-item" href="#">Action</a></li>
      <li><a class="dropdown-item" href="#">Another action</a></li>
    </ul>
  </li>
</ul>
```

6. Tabbed Content (JavaScript)

To make the tabs actually "work" and swap content sections, you use the `data-bs-toggle="tab"` attribute along with `.tab-content` and `.tab-pane` containers.

HTML

```
<ul class="nav nav-tabs" id="myTab" role="tablist">
  <li class="nav-item">
    <button class="nav-link active" data-bs-toggle="tab" data-bs-
target="#home" type="button">Home</button>
  </li>
  <li class="nav-item">
    <button class="nav-link" data-bs-toggle="tab" data-bs-target="#profile"
type="button">Profile</button>
  </li>
</ul>

<div class="tab-content" id="myTabContent">
  <div class="tab-pane fade show active" id="home">Home content...</div>
  <div class="tab-pane fade" id="profile">Profile content...</div>
</div>
```

Bootstrap 5 Navbars

The **Navbar** is one of the most complex and powerful components in Bootstrap 5. it serves as a responsive navigation header that includes support for branding, navigation links, forms, and dropdowns. By default, it is mobile-friendly—collapsing into a "hamburger" menu on smaller screens.

1. Basic Navbar Structure

A standard navbar requires a specific wrapper and several inner elements.

HTML

```
<nav class="navbar navbar-expand-lg navbar-light bg-light">
  <div class="container-fluid">
    <a class="navbar-brand" href="#">Navbar</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
data-bs-target="#navbarNav">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navbarNav">
      <ul class="navbar-nav">
        <li class="nav-item">
          <a class="nav-link active" href="#">Home</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="#">Features</a>
        </li>
      </ul>
    </div>
  </div>
</nav>
```

2. Key Classes Explained

Class	Purpose
<code>.navbar</code>	The base wrapper for the component.
<code>.navbar-expand-lg</code>	Determines at which breakpoint the menu "unfolds" (sm, md, lg, xl).
<code>.navbar-brand</code>	Used for your project name or logo.
<code>.navbar-toggler</code>	The button that appears on mobile to toggle the menu.
<code>.collapse.navbar-collapse</code>	Groups the links to be hidden/shown on mobile.
<code>.navbar-nav</code>	A specialized version of the Nav component for full-height headers.

Export to Sheets

3. Color Schemes

Bootstrap 5 makes it easy to theme your navbar using background utilities.

- **Light Navbar:** Use `.navbar-light` with `.bg-light`.
- **Dark Navbar:** Use `.navbar-dark` with `.bg-dark` or `.bg-primary`.

HTML

```
<nav class="navbar navbar-dark bg-dark">  
  </nav>
```

4. Placement & Fixes

You can control how the navbar sits on the page:

- **Fixed Top:** `.fixed-top` (Stays at the top while scrolling).
- **Fixed Bottom:** `.fixed-bottom` (Stays at the bottom).
- **Sticky Top:** `.sticky-top` (Stays at the top only once you scroll past it).

Note: If you use `.fixed-top`, you will need to add padding to the top of your `<body>` tag so the navbar doesn't cover your content.

5. Adding Forms and Text

Navbars often include search bars or "Welcome, User" text.

HTML

```
<nav class="navbar navbar-light bg-light">
  <div class="container-fluid">
    <span class="navbar-text">Navbar text with an inline element</span>
    <form class="d-flex">
      <input class="form-control me-2" type="search" placeholder="Search">
      <button class="btn btn-outline-success" type="submit">Search</button>
    </form>
  </div>
</nav>
```

6. Responsive Behavior

The `.navbar-expand-{breakpoint}` class is crucial.

- If you use `.navbar-expand-md`, the navbar will show the hamburger icon on mobile and small tablets, but switch to a full horizontal menu on medium screens and up.

Bootstrap 5 Carousel

The **Carousel** in Bootstrap 5 is a slideshow component for cycling through elements—typically images or text slides—like a gallery. It includes support for previous/next controls and indicators.

1. Basic Carousel

The most basic carousel requires a wrapper with a unique `id`, the `.carousel` and `.slide` classes, and an inner container for the slides.

HTML

```
<div id="carouselExample" class="carousel slide">
  <div class="carousel-inner">
    <div class="carousel-item active">
      
    </div>
    <div class="carousel-item">
      
    </div>
  </div>
</div>
```

- **.active:** Exactly one slide must have this class, or the carousel will not be visible.

- `.d-block.w-100`: These utility classes prevent browser default image alignment issues and ensure the image fills the width of the carousel.
-

2. Controls and Indicators

To allow users to navigate, you add control buttons and indicators (small dots or bars at the bottom).

Controls (Prev/Next)

HTML

```
<button class="carousel-control-prev" type="button" data-bs-  
target="#carouselExample" data-bs-slide="prev">  
  <span class="carousel-control-prev-icon" aria-hidden="true"></span>  
  <span class="visually-hidden">Previous</span>  
</button>  
<button class="carousel-control-next" type="button" data-bs-  
target="#carouselExample" data-bs-slide="next">  
  <span class="carousel-control-next-icon" aria-hidden="true"></span>  
  <span class="visually-hidden">Next</span>  
</button>
```

Indicators

HTML

```
<div class="carousel-indicators">  
  <button type="button" data-bs-target="#carouselExample" data-bs-slide-  
to="0" class="active"></button>  
  <button type="button" data-bs-target="#carouselExample" data-bs-slide-  
to="1"></button>  
</div>
```

3. Captions

You can add labels or descriptions to each slide using the `.carousel-caption` class inside any `.carousel-item`. These are automatically hidden on smaller screens and shown on medium devices and up.

HTML

```
<div class="carousel-item">  
    
  <div class="carousel-caption d-none d-md-block">  
    <h5>First Slide Label</h5>  
    <p>Some representative placeholder content for the first slide.</p>  
</div>
```

</div>

4. Crossfade Effect

By default, the carousel uses a sliding transition. To use a fading transition instead, add the `.carousel-fade` class to the main carousel container.

HTML

```
<div id="carouselExampleFade" class="carousel slide carousel-fade">
  ...
</div>
```

5. Variations and Timing

Feature	Class / Attribute	Description
Dark Variant	<code>.carousel-dark</code>	Inverts colors for controls and indicators (best for light backgrounds).
Interval	<code>data-bs-interval="2000"</code>	Changes the amount of time between slides (in milliseconds).
Autoplay	<code>data-bs-ride="carousel"</code>	Tells the carousel to start animating immediately on page load.
Disable Touch	<code>data-bs-touch="false"</code>	Disables swiping on touchscreen devices.

Export to Sheets

6. Sizing Tips

Carousels do not have a fixed height by default; they take the height of the largest image. For a professional look:

1. **Use consistent image sizes:** Ensure all images have the same aspect ratio.
2. **Object Fit:** If images vary, use custom CSS like `object-fit: cover;` and a fixed height on the `.carousel-item`.

Bootstrap 5 Tooltip

Tooltips in Bootstrap 5 are small, interactive popups that appear when a user hovers over or clicks an element. They are commonly used to provide extra context for icons or buttons without cluttering the UI.

1. Important: Initialization Required

Unlike many other Bootstrap components (like Modals or Alerts), **Tooltips are not enabled by default** for performance reasons. You must initialize them using JavaScript before they will work.

Add this script to your project (usually at the bottom of your HTML):

JavaScript

```
const tooltipTriggerList = document.querySelectorAll('[data-bs-toggle="tooltip"]')
const tooltipList = [...tooltipTriggerList].map(tooltipTriggerEl => new bootstrap.Tooltip(tooltipTriggerEl))
```

2. Basic Tooltip Structure

To create a tooltip, you add the `data-bs-toggle="tooltip"` attribute and a `title` attribute, which contains the text you want to display.

HTML

```
<button type="button" class="btn btn-secondary" data-bs-toggle="tooltip" data-bs-placement="top" title="Tooltip on top">
  Hover over me
</button>
```

3. Placement Options

You can control where the tooltip appears relative to the element using the `data-bs-placement` attribute.

Placement	Attribute	Effect
Top	<code>data-bs-placement="top"</code>	Appears above the element.
Bottom	<code>data-bs-placement="bottom"</code>	Appears below the element.
Left	<code>data-bs-placement="left"</code>	Appears to the left.
Right	<code>data-bs-placement="right"</code>	Appears to the right.
Export to Sheets		

4. Customizing Tooltips

Bootstrap 5 allows you to customize the look and behavior of tooltips using data attributes.

- **HTML Support:** By default, tooltips only show plain text. To use HTML (like bold text or icons), add `data-bs-html="true"`.
- **Custom Class:** You can add your own CSS class to a tooltip using `data-bs-custom-class="custom-tooltip"`.
- **Animation:** To disable the fade-in effect, use `data-bs-animation="false"`.

HTML

```
<button type="button" class="btn btn-secondary"
  data-bs-toggle="tooltip"
  data-bs-html="true"
  title="<em>Tooltip</em> <u>with</u> <b>HTML</b>">
  Tooltip with HTML
</button>
```

5. Tooltips on Disabled Elements

Elements with the `disabled` attribute do not trigger hover events, so tooltips won't show. To fix this, wrap the disabled element in a `<span>` or `<div>` and apply the tooltip to the wrapper instead.

HTML

```
<span class="d-inline-block" tabindex="0" data-bs-toggle="tooltip"
  title="Disabled tooltip">
  <button class="btn btn-primary" type="button" disabled>Disabled
  button</button>
</span>
```

6. Popovers vs. Tooltips

Feature	Tooltip	Popover
Content	Text only (usually)	Title + Body content
Trigger	Hover/Focus	Click (usually)
Purpose	Brief labels	Detailed information
Export to Sheets		
Bootstrap 5 Popover		

Popovers in Bootstrap 5 are similar to tooltips but are designed to hold more content, such as a title and a body of text. They are usually triggered by a click rather than a hover, making them ideal for "More Info" buttons or help icons.

1. Important: Initialization Required

Just like tooltips, **Popovers are not enabled by default**. You must initialize them using JavaScript. Add this script to your project:

JavaScript

```
const popoverTriggerList = document.querySelectorAll('[data-bs-toggle="popover"]')
const popoverList = [...popoverTriggerList].map(popoverTriggerEl => new bootstrap.Popover(popoverTriggerEl))
```

2. Basic Popover Structure

To create a popover, use the `data-bs-toggle="popover"` attribute. You can provide a `title` (the header) and `data-bs-content` (the main body).

HTML

```
<button type="button" class="btn btn-lg btn-danger"
        data-bs-toggle="popover"
        title="Popover title"
        data-bs-content="And here's some amazing content. It's very engaging.
Right?">
  Click to toggle popover
</button>
```

3. Four Directions

You can position the popover using the `data-bs-placement` attribute.

- **Top:** `data-bs-placement="top"`
 - **Bottom:** `data-bs-placement="bottom"`
 - **Left:** `data-bs-placement="left"`
 - **Right:** `data-bs-placement="right"`
-

4. Dismiss on Next Click

By default, a popover stays open until you click the trigger button again. To make it close when the user clicks *anywhere else* on the page, use the `focus` trigger.

Note: For this to work across all browsers, you must use the `<a>` tag (not `<button>`) and include a `tabindex` attribute.

HTML

```
<a tabindex="0" class="btn btn-lg btn-danger" role="button"
  data-bs-toggle="popover"
  data-bs-trigger="focus"
  title="Dismissible popover"
  data-bs-content="Click anywhere else on the page to close me!">
  Dismissible popover
</a>
```

5. Key Configuration Options

You can pass these options via data attributes to change how the popover behaves:

Attribute	Default	Description
data-bs-animation	true	Apply a CSS fade transition to the popover.
data-bs-html	false	Set to <code>true</code> to allow HTML inside the content.
data-bs-delay	0	Delay showing/hiding (e.g., <code>data-bs-delay="500"</code>).
data-bs-trigger	click	How the popover is triggered (<code>click</code> , <code>hover</code> , <code>focus</code>).

[Export to Sheets](#)

6. Disabled Elements

Just like tooltips, popovers on disabled elements won't trigger. Wrap the disabled element in a `<div>` or `<span>` and put the popover on that wrapper instead.

HTML

```
<span class="d-inline-block" tabindex="0" data-bs-toggle="popover" data-bs-
content="Disabled popover">
  <button class="btn btn-primary" type="button" disabled>Disabled
button</button>
</span>
```

Bootstrap 5 Toasts

Toasts in Bootstrap 5 are lightweight, customizable notifications designed to mimic the push notifications popularized by mobile and desktop operating systems. They are "snackbars" that typically appear in a corner of the screen to give quick feedback.

1. Important: Initialization Required

Like tooltips and popovers, **Toasts are opt-in for performance reasons**. You must initialize them with JavaScript to make them appear.

JavaScript

```
const toastElList = document.querySelectorAll('.toast')
const toastList = [...toastElList].map(toastEl => new
bootstrap.Toast(toastEl))

// To show a specific toast:
// toastList[0].show();
```

2. Basic Toast Structure

A toast is composed of a main container, a header (optional but recommended), and a body.

HTML

```
<div class="toast" role="alert" aria-live="assertive" aria-atomic="true">
  <div class="toast-header">
    <strong class="me-auto">Bootstrap</strong>
    <small>11 mins ago</small>
    <button type="button" class="btn-close" data-bs-dismiss="toast" aria-
label="Close"></button>
  </div>
  <div class="toast-body">
    Hello, world! This is a toast message.
  </div>
</div>
```

3. Positioning Toasts

Toasts do not have a default position; they simply appear in the DOM where you place them. To make them "float" in a corner (like the top-right), use the **Layout** and **Positioning** utilities.

HTML

```
<div class="toast-container position-fixed bottom-0 end-0 p-3">
  <div id="liveToast" class="toast" role="alert" aria-live="assertive" aria-
atomic="true">
    <div class="toast-header">
      <strong class="me-auto">Notification</strong>
      <button type="button" class="btn-close" data-bs-dismiss="toast" aria-
label="Close"></button>
    </div>
    <div class="toast-body">
      Your settings have been saved.
    </div>
  </div>
</div>
```

- **.toast-container:** Helps manage multiple toasts stacking on top of each other.
- **.position-fixed:** Ensures the notification stays in place even if the user scrolls.

4. Customizing Appearance

You can use Bootstrap's color utilities to change the look of your toasts. For a "Success" or "Error" toast, apply background and text color classes to the main `.toast` div.

HTML

```
<div class="toast align-items-center text-white bg-primary border-0"
role="alert">
  <div class="d-flex">
    <div class="toast-body">
      Hello, world! This is a primary toast.
    </div>
    <button type="button" class="btn-close btn-close-white me-2 m-auto" data-
bs-dismiss="toast"></button>
  </div>
</div>
```

5. Key Options

You can configure toast behavior using data attributes or JavaScript:

Attribute	Default	Description
<code>data-bs-animation</code>	<code>true</code>	Apply a CSS fade transition to the toast.
<code>data-bs-autohide</code>	<code>true</code>	Automatically hide the toast after a delay.
<code>data-bs-delay</code>	<code>5000</code>	Delay in milliseconds before hiding the toast.
Export to Sheets		

6. Accessibility Notes

Toasts are intended to be "small interruptions." To make them accessible:

- `role="alert"`: Use for important, time-sensitive information (errors).
- `role="status"`: Use for less urgent information (success messages).
- `aria-live="assertive"`: For alerts (interrupts screen readers).
- `aria-live="polite"`: For status updates (waits for screen reader to finish current task).

Bootstrap 5 Scrollspy

Scrollspy in Bootstrap 5 is a navigation mechanism that automatically updates links in a navigation list based on the user's current scroll position. It is commonly used in one-

page websites or long documentation pages to show which section is currently being viewed.

1. How it Works

Scrollspy requires two main components:

1. **The Navigation:** A `nav` or `list-group` with links that point to IDs on the page (e.g., `<a href="#section1">`).
 2. **The Scroll Target:** The element being scrolled (usually the `<body>` or a container with `overflow-y: scroll`).
-

2. Basic Requirements

To set up Scrollspy, you add specific data attributes to the **scrollable element** (the container with the content):

- `data-bs-spy="scroll"`: Activates the Scrollspy behavior.
- `data-bs-target="#nav-id"`: Tells Scrollspy which navigation component to update.
- `data-bs-smooth-scroll="true"`: (Optional) Enables smooth scrolling when clicking links.

HTML

```
<nav id="navbar-example" class="navbar bg-light px-3 mb-3">
  <a class="navbar-brand" href="#">Project Name</a>
  <ul class="nav nav-pills">
    <li class="nav-item"><a class="nav-link"
href="#scrollspyHeading1">First</a></li>
    <li class="nav-item"><a class="nav-link"
href="#scrollspyHeading2">Second</a></li>
  </ul>
</nav>

<div data-bs-spy="scroll" data-bs-target="#navbar-example" data-bs-smooth-
scroll="true" class="scrollspy-example" tabindex="0">
  <h4 id="scrollspyHeading1">First heading</h4>
  <p>...</p>
  <h4 id="scrollspyHeading2">Second heading</h4>
  <p>...</p>
</div>
```

3. CSS Prerequisites

If you are spying on an element other than the `<body>`, you **must** set a height and an overflow property in your CSS for that container:

CSS

```
.scrollspy-example {  
  position: relative;  
  height: 350px;  
  overflow-y: scroll;  
}
```

4. Nested Navigation

Scrollspy also works with nested navs. If you have a dropdown or a sub-navigation, the parent link will stay highlighted while the user is viewing any of the sub-sections.

HTML

```
<ul class="nav flex-column">  
  <li class="nav-item">  
    <a class="nav-link" href="#item-1">Item 1</a>  
    <nav class="nav flex-column ms-3">  
      <a class="nav-link" href="#item-1-1">Item 1-1</a>  
    </nav>  
  </li>  
</ul>
```

5. Key Options & Attributes

Attribute	Description
<code>data-bs-root-margin</code>	Adjusts the margin for detecting when an element is "in view" (e.g., <code>0px 0px -40%</code>).
<code>data-bs-threshold</code>	An array of values (0 to 1.0) indicating at what percentage of the target's visibility the callback should be executed.
<code>data-bs-smooth-scroll</code>	Set to <code>true</code> to animate the transition when a user clicks a nav link.
Export to Sheets	

6. Pro-Tip: Refreshing Scrollspy

If you dynamically add or remove content from your page while Scrollspy is active, you may need to refresh it so it calculates the new positions correctly:

JavaScript

```
const scrollSpy = bootstrap.ScrollSpy.getInstance(document.body);
scrollSpy.refresh();
```

Bootstrap 5 Offcanvas

Offcanvas in Bootstrap 5 is a sidebar component that can be toggled to appear from the left, right, top, or bottom edge of the viewport. It is commonly used for mobile navigation menus, shopping carts, or advanced filter panels.

Unlike Modals, Offcanvas allows the background content to remain visible (though often dimmed by a backdrop).

1. Basic Structure

An offcanvas requires a **trigger** (button or link) and the **offcanvas container**.

- **The Trigger:** Needs `data-bs-toggle="offcanvas"` and `data-bs-target="#id"`.
- **The Container:** Needs the `.offcanvas` class and a placement class (e.g., `.offcanvas-start`).

HTML

```
<button class="btn btn-primary" type="button" data-bs-toggle="offcanvas"
data-bs-target="#offcanvasExample">
  Open Sidebar
</button>

<div class="offcanvas offcanvas-start" tabindex="-1" id="offcanvasExample"
aria-labelledby="offcanvasExampleLabel">
  <div class="offcanvas-header">
    <h5 class="offcanvas-title" id="offcanvasExampleLabel">Menu</h5>
    <button type="button" class="btn-close" data-bs-dismiss="offcanvas" aria-
label="Close"></button>
  </div>
  <div class="offcanvas-body">
    Your content goes here. You can place lists, forms, or navigation links.
  </div>
</div>
```

2. Placement Options

You can change which side the sidebar appears from by changing the modifier class on the main `.offcanvas` div:

Class	Position
<code>.offcanvas-start</code>	Left side (in LTR languages).
<code>.offcanvas-end</code>	Right side (in LTR languages).
<code>.offcanvas-top</code>	Top of the viewport.
<code>.offcanvas-bottom</code>	Bottom of the viewport.
Export to Sheets	

3. Backdrops and Scrolling

You can control how the rest of the page behaves when the offcanvas is open using data attributes:

- **Enable Body Scrolling:** Add `data-bs-scroll="true"` to the trigger or container to allow the user to scroll the main page while the sidebar is open.
- **Disable Backdrop:** Add `data-bs-backdrop="false"` if you don't want the dark overlay behind the sidebar.

HTML

```
<button class="btn btn-primary" type="button" data-bs-toggle="offcanvas"
data-bs-target="#offcanvasScrolling" data-bs-scroll="true" data-bs-
backdrop="false">
  Enable body scrolling
</button>
```

4. Responsive Offcanvas

In Bootstrap 5.2+, you can make an offcanvas responsive. This means it can behave like a normal sidebar on large screens but turn into a hidden offcanvas on mobile.

Use `.offcanvas-{breakpoint}` (e.g., `.offcanvas-lg`):

- On screens smaller than **lg**, the content is hidden and requires a toggle.
 - On screens **lg** and larger, the content is shown as a standard part of the page layout.
-

5. JavaScript Control

If you want to open or close the offcanvas programmatically (for example, after a form submission), use the Bootstrap constructor:

JavaScript

```
const myOffcanvas = document.getElementById('offcanvasExample');
const bsOffcanvas = new bootstrap.Offcanvas(myOffcanvas);

// Show the sidebar
bsOffcanvas.show();

// Hide the sidebar
bsOffcanvas.hide();
```

Accessibility Check

- Ensure every offcanvas has a unique `id`.
- Include a `btn-close` button inside the header so keyboard users can easily exit.
- Use `aria-labelledby` on the container to point to the `offcanvas-title`.

Bootstrap 5 Utilities

Utilities in Bootstrap 5 are low-level CSS classes that allow you to style elements directly in your HTML without writing custom CSS. They follow a specific naming convention (e.g., `property-sides-size`) and are built with mobile-first breakpoints in mind.

1. Spacing (Margin & Padding)

Spacing utilities use a scale from 0 to 5 (plus `auto`).

- **m** for margin, **p** for padding.
- **t** (top), **b** (bottom), **s** (start/left), **e** (end/right), **x** (left & right), **y** (top & bottom).

HTML

```
<div class="mt-3">Margin top 1rem</div>
<div class="px-5">Padding left & right 3rem</div>
<div class="mx-auto" style="width: 200px;">Centered block</div>
```

2. Text Utilities

Control alignment, wrapping, weight, and color easily.

Category	Classes
Alignment	<code>.text-center</code> , <code>.text-start</code> , <code>.text-end</code>

Category	Classes
Wrapping	<code>.text-wrap</code> , <code>.text-nowrap</code> , <code>.text-break</code>
Transform	<code>.text-lowercase</code> , <code>.text-uppercase</code> , <code>.text-capitalize</code>
Weight	<code>.fw-bold</code> , <code>.fw-light</code> , <code>.fst-italic</code>
Decoration	<code>.text-decoration-none</code> , <code>.text-decoration-underline</code>
Export to Sheets	

3. Colors & Backgrounds

Apply the standard theme colors to text or backgrounds.

- **Text:** `.text-primary`, `.text-success`, `.text-danger`, `.text-muted`.
- **Background:** `.bg-primary`, `.bg-dark`, `.bg-light`, `.bg-transparent`.
- **Opacity:** `.bg-opacity-50`, `.text-opacity-75`.

HTML

```
<p class="text-primary bg-light p-2">Blue text on light gray background</p>
```

4. Display & Visibility

Control when elements appear or how they behave in the document flow.

- **Display:** `.d-none`, `.d-inline`, `.d-block`, `.d-flex`, `.d-grid`.
 - **Responsive:** `.d-md-none` (hidden only on medium screens and up).
 - **Visibility:** `.visible` or `.invisible` (hides the element but keeps its space in the layout).
-

5. Flexbox Utilities

Bootstrap 5 is built on Flexbox. You can manage layouts without writing a single line of `display: flex`.

- **Direction:** `.flex-row`, `.flex-column`.
- **Justify:** `.justify-content-start`, `.justify-content-center`, `.justify-content-between`.
- **Align:** `.align-items-center`, `.align-self-end`.
- **Fill/Grow:** `.flex-fill`, `.flex-grow-1`.

HTML

```
<div class="d-flex justify-content-between align-items-center">
```

```
<div>Item 1</div>
<div>Item 2</div>
</div>
```

6. Sizing (Width & Height)

Set the width or height of an element relative to its parent.

- **Width:** `.w-25`, `.w-50`, `.w-75`, `.w-100`, `.vw-100` (viewport width).
 - **Height:** `.h-25`, `.h-50`, `.h-75`, `.h-100`, `.vh-100` (viewport height).
-

7. Borders & Shadows

Add quick styling to containers.

- **Borders:** `.border`, `.border-top-0`, `.border-primary`, `.border-3` (thickness).
- **Radius:** `.rounded`, `.rounded-circle`, `.rounded-pill`, `.rounded-0` (sharp corners).
- **Shadows:** `.shadow-none`, `.shadow-sm`, `.shadow`, `.shadow-lg`.

HTML

```
<div class="shadow-lg p-3 mb-5 bg-body rounded">Large Shadow Box</div>
```

8. Stacks (New in v5)

Stacks are shorthand for vertical or horizontal flex containers with gaps.

- **Vertical:** `.vstack`
- **Horizontal:** `.hstack`
- **Gap:** Use `.gap-1` to `.gap-5` to space items.

HTML

```
<div class="vstack gap-3">
  <div class="bg-light border">First item</div>
  <div class="bg-light border">Second item</div>
</div>
```

Bootstrap 5 Dark Mode

Bootstrap 5 introduced native **Dark Mode** support in version 5.3. Unlike previous versions where you had to manually swap classes, it now uses **Color Modes** powered

by CSS variables. This makes it incredibly easy to toggle your entire site or specific sections to a dark theme.

1. Enabling Dark Mode (The Global Way)

The simplest way to turn on dark mode for your entire page is to add the `data-bs-theme="dark"` attribute to the `<html>` or `<body>` tag.

HTML

```
<!doctype html>
<html lang="en" data-bs-theme="dark">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <link href="bootstrap.min.css" rel="stylesheet">
    <title>Bootstrap Dark Mode</title>
  </head>
  <body>
    <h1>Hello, dark world!</h1>
  </body>
</html>
```

2. Partial Dark Mode (Sectional)

You don't have to make the whole site dark. You can apply the theme to specific containers, like a single card or a sidebar, while keeping the rest of the page light.

HTML

```
<div class="card" data-bs-theme="dark">
  <div class="card-body">
    <h5 class="card-title">Dark Card</h5>
    <p>This card is dark even if the page is light.</p>
    <button class="btn btn-primary">Action</button>
  </div>
</div>
```

3. Dark Mode Specific Colors

Bootstrap 5.3+ includes new CSS variables and utility classes that automatically adjust based on the theme. Instead of hardcoding colors, use the **Emphasis** and **Subtle** utilities.

Light Mode Result	Dark Mode Result	Class
Black Text	White Text	<code>.text-body</code>
Gray Text	Light Gray Text	<code>.text-body-secondary</code>
White BG	Dark Gray BG	<code>.bg-body</code>
Light Blue	Dark Navy Blue	<code>.bg-primary-subtle</code>
Export to Sheets		

4. Building a Theme Toogle (JavaScript)

To allow users to switch between light and dark modes, you need a small bit of JavaScript to update the `data-bs-theme` attribute.

JavaScript

```
const toggleTheme = () => {
  const htmlElement = document.documentElement;
  if (htmlElement.getAttribute('data-bs-theme') === 'dark') {
    htmlElement.setAttribute('data-bs-theme', 'light');
  } else {
    htmlElement.setAttribute('data-bs-theme', 'dark');
  }
}
```

5. Automatic (System) Detection

Most modern browsers allow you to detect if a user has "Dark Mode" enabled in their Windows or macOS settings. You can sync your website to match their system preference automatically.

JavaScript

```
// Check for system preference
if (window.matchMedia('(prefers-color-scheme: dark)').matches) {
  document.documentElement.setAttribute('data-bs-theme', 'dark');
}
```

6. Component Specifics

Most components automatically adapt when the theme changes:

- **Dropdowns:** Menus turn dark with light text.
- **Modals:** Backgrounds darken and borders soften.
- **Forms:** Inputs get darker backgrounds and lighter borders to maintain contrast.

- **Navbars:** If you were using `.navbar-dark`, it is still supported, but using `data-bs-theme="dark"` on the navbar is now the preferred way.
-

Quick Summary of Benefits

- **No more custom CSS:** You don't need to write `@media (prefers-color-scheme: dark)` manually.
- **Improved Contrast:** Bootstrap's dark mode colors are accessibility-tested.
- **Consistency:** All components (Buttons, Cards, Forms) look uniform in the dark theme.

Bootstrap 5 Flex

Bootstrap 5 is built entirely on **Flexbox**, meaning almost every component (Navbars, Card groups, etc.) uses it under the hood. Bootstrap provides a suite of shorthand utilities so you can manage layout, alignment, and sizing without writing a single line of custom CSS.

1. Enable Flexbox Behaviors

To turn an element into a flex container, use the `.d-flex` (block-level) or `.d-inline-flex` (inline-level) utilities.

HTML

```
<div class="d-flex p-2 bg-light border">I'm a flexbox container!</div>
```

2. Direction

Use direction utilities to set how items are placed in the flex container.

- **Row (Default):** `.flex-row` (Horizontal)
- **Row Reverse:** `.flex-row-reverse` (Starts from the right)
- **Column:** `.flex-column` (Vertical stack)
- **Column Reverse:** `.flex-column-reverse` (Vertical stack from bottom to top)

HTML

```
<div class="d-flex flex-column">  
  <div>Item 1</div>  
  <div>Item 2</div>
```

</div>

3. Justify Content (Horizontal Alignment)

Use these to change the alignment of flex items on the **main axis** (horizontally if using `flex-row`).

Class	Description
<code>.justify-content-start</code>	Items at the beginning (Default).
<code>.justify-content-end</code>	Items at the end.
<code>.justify-content-center</code>	Items in the center.
<code>.justify-content-between</code>	Items at the edges with even space between.
<code>.justify-content-around</code>	Even space around each item.

Export to Sheets

4. Align Items (Vertical Alignment)

Use these to change the alignment of flex items on the **cross axis** (vertically if using `flex-row`).

- **Start:** `.align-items-start`
 - **End:** `.align-items-end`
 - **Center:** `.align-items-center`
 - **Baseline:** `.align-items-baseline`
 - **Stretch:** `.align-items-stretch` (Default)
-

5. Align Self

You can override the `align-items` for a **specific** child item using `.align-self-*`.

HTML

```
<div class="d-flex" style="height: 100px;">
  <div class="p-2 border">Item 1</div>
  <div class="align-self-center p-2 border">I'm centered!</div>
  <div class="p-2 border">Item 3</div>
</div>
```

6. Wrap and Order

- **Wrap:** By default, flex items will try to fit onto one line. Use `.flex-wrap` to allow them to wrap onto multiple lines.
- **Order:** Change the visual order of items without changing the HTML. Use `.order-0` through `.order-5`.

HTML

```
<div class="d-flex">  
  <div class="order-3">First in HTML, Third Visually</div>  
  <div class="order-1">Second in HTML, First Visually</div>  
</div>
```

7. Grow and Shrink

- **Grow:** Use `.flex-grow-1` to force an item to take up all the remaining space.
- **Shrink:** Use `.flex-shrink-1` to allow an item to shrink if necessary.

HTML

```
<div class="d-flex">  
  <div class="p-2 w-100">Flex item</div> <div class="p-2 flex-shrink-1">Fixed</div> </div>
```

8. Auto Margins

This is a "secret weapon" in Bootstrap Flex. Using `.ms-auto` (`margin-start: auto`) or `.me-auto` (`margin-end: auto`) inside a flex container will push the items apart. This is how navbars push the "Login" button to the far right.

HTML

```
<div class="d-flex">  
  <div class="p-2">Brand</div>  
  <div class="ms-auto p-2">Login</div> </div>
```

Bootstrap 5 Forms

Bootstrap 5 Forms are designed to be modern, accessible, and easy to customize. By default, Bootstrap applies its own styles to standard HTML form controls, ensuring they look consistent across all browsers and devices.

1. Basic Form Structure

The foundation of a Bootstrap form is the `.form-label` and the `.form-control`. Unlike older versions, Bootstrap 5 does not require a specific wrapper like `.form-group`, though many developers still use margin utilities (like `.mb-3`) to create spacing between fields.

HTML

```
<div class="mb-3">
  <label for="exampleInputEmail1" class="form-label">Email address</label>
  <input type="email" class="form-control" id="exampleInputEmail1"
placeholder="name@example.com">
</div>
<div class="mb-3">
  <label for="exampleFormControlTextarea1" class="form-
label">Comments</label>
  <textarea class="form-control" id="exampleFormControlTextarea1"
rows="3"></textarea>
</div>
```

2. Form Controls & Sizing

You can change the size of your inputs using modifier classes.

- **Large:** `.form-control-lg`
- **Small:** `.form-control-sm`
- **Read-only:** Add the `readonly` attribute to prevent modification while keeping the input style.

3. Checks, Radios, and Switches

Bootstrap 5 uses completely custom checkboxes and radio buttons to ensure they look identical across different operating systems.

- **Checkboxes:** Use `.form-check-input` and `.form-check-label`.
- **Switches:** Add the `.form-switch` class to a `.form-check` wrapper to turn a checkbox into a toggle switch.

HTML

```
<div class="form-check form-switch">
  <input class="form-check-input" type="checkbox"
id="flexSwitchCheckDefault">
  <label class="form-check-label" for="flexSwitchCheckDefault">Enable
Notifications</label>
</div>
```

4. Range and File Inputs

- **Range:** Use `.form-range` for a horizontal slider.
 - **File Input:** Use `.form-control` on an `<input type="file">`. Bootstrap handles the "Browse" button styling automatically.
-

5. Input Groups

Input groups allow you to prepend or append text, buttons, or checkboxes to your inputs. Use the `.input-group` wrapper and `.input-group-text` for the labels.

HTML

```
<div class="input-group mb-3">
  <span class="input-group-text" id="basic-addon1">@</span>
  <input type="text" class="form-control" placeholder="Username">
</div>

<div class="input-group mb-3">
  <input type="text" class="form-control" placeholder="Recipient's username">
  <button class="btn btn-outline-secondary" type="button">Search</button>
</div>
```

6. Floating Labels

Floating labels create an elegant effect where the label "floats" above the input field once the user starts typing or the field is focused.

Note: For floating labels to work, the `<input>` must come **before** the `<label>`, and the input **must** have a `placeholder` (even if it's empty).

HTML

```
<div class="form-floating mb-3">
  <input type="email" class="form-control" id="floatingInput"
placeholder="name@example.com">
  <label for="floatingInput">Email address</label>
</div>
```

7. Form Layout (Grid)

For more complex layouts, you can use the **Bootstrap Grid System** inside your forms. Use the `.row` and `.col` classes to place inputs side-by-side.

HTML

```
<div class="row g-3">
  <div class="col">
    <input type="text" class="form-control" placeholder="First name">
  </div>
  <div class="col">
    <input type="text" class="form-control" placeholder="Last name">
  </div>
</div>
```

8. Validation

Bootstrap provides easy-to-use validation styles. You can apply `.is-valid` or `.is-invalid` classes manually, or use browser-native validation with the `.needs-validation` class on the `<form>`.

- **Valid Feedback:** `.valid-feedback`
- **Invalid Feedback:** `.invalid-feedback`

HTML

```
<div class="mb-3">
  <label class="form-label">Username</label>
  <input type="text" class="form-control is-invalid" required>
  <div class="invalid-feedback">Please choose a username.</div>
</div>
```

Bootstrap 5 Select

In Bootstrap 5, the **Select** menu is a custom-styled version of the native `<select>` element. It ensures that dropdown menus look consistent across different browsers and operating systems while remaining fully accessible.

1. Basic Select

To apply Bootstrap styling, add the `.form-select` class to your `<select>` element. This replaces the default browser appearance with a custom arrow and consistent padding.

HTML

```
<select class="form-select" aria-label="Default select example">
  <option selected>Open this select menu</option>
  <option value="1">One</option>
  <option value="2">Two</option>
  <option value="3">Three</option>
</select>
```

2. Sizing

You can quickly change the height of your select menus using modifier classes to match the rest of your form controls.

- **Large:** `.form-select-lg`
- **Small:** `.form-select-sm`

HTML

```
<select class="form-select form-select-lg mb-3">
  <option selected>Large select menu</option>
</select>
```

```
<select class="form-select form-select-sm">
  <option selected>Small select menu</option>
</select>
```

3. Multiple Select

If you want to allow users to select more than one option, add the `multiple` attribute. The `.form-select` class will automatically adjust the height to show multiple rows.

HTML

```
<select class="form-select" multiple aria-label="multiple select example">
  <option selected>Open this select menu</option>
  <option value="1">One</option>
  <option value="2">Two</option>
</select>
```

4. Size Attribute

You can use the `size` attribute to determine how many options should be visible at once without making it a "multiple" choice menu.

HTML

```
<select class="form-select" size="3" aria-label="size 3 select example">
  <option selected>Open this select menu</option>
  <option value="1">One</option>
  <option value="2">Two</option>
  <option value="3">Three</option>
</select>
```

5. Disabled State

Add the `disabled` boolean attribute to the select to prevent user interaction and make it appear grayed out.

HTML

```
<select class="form-select" disabled aria-label="Disabled select example">
  <option selected>You cannot click this</option>
</select>
```

6. Validation States

Just like other form controls, Select menus support validation classes to provide feedback to the user.

- **Valid:** `.is-valid` (Green border and checkmark)
- **Invalid:** `.is-invalid` (Red border and exclamation icon)

HTML

```
<select class="form-select is-invalid" required>
  <option selected disabled value="">Choose...</option>
  <option>...</option>
</select>
<div class="invalid-feedback">
  Please select a valid state.
</div>
```

7. Floating Labels with Select

Select menus also work with Bootstrap's **Floating Labels**. To use this, wrap the `.form-select` and a `<label>` inside a `.form-floating` container.

HTML

```
<div class="form-floating">
  <select class="form-select" id="floatingSelect" aria-label="Floating label
select example">
    <option selected>Open this select menu</option>
    <option value="1">One</option>
    <option value="2">Two</option>
  </select>
  <label for="floatingSelect">Works with selects</label>
</div>
```

Bootstrap 5 Checkboxes and Radio buttons

In Bootstrap 5, **Checkboxes** and **Radios** are completely overhauled to use custom CSS. This ensures they look identical across all browsers and operating systems (like iOS, Android, and Windows) while remaining fully accessible and supporting high-contrast modes.

1. Basic Structure

To style a checkbox or radio, wrap the `<input>` and `<label>` in a `<div>` with the `.form-check` class.

- **Input:** Add the `.form-check-input` class.
- **Label:** Add the `.form-check-label` class.

Checkbox Example

Checkboxes are used for selecting one or more options in a list.

HTML

```
<div class="form-check">
  <input class="form-check-input" type="checkbox" value=""
id="flexCheckDefault">
  <label class="form-check-label" for="flexCheckDefault">
    Default checkbox
  </label>
</div>
```

Radio Example

Radios are used for selecting only one option from a list. Ensure all items in a group share the same `name` attribute.

HTML

```
<div class="form-check">
  <input class="form-check-input" type="radio" name="flexRadioDefault"
id="flexRadioDefault1">
  <label class="form-check-label" for="flexRadioDefault1">
    Default radio
  </label>
</div>
```

2. Indeterminate State

Checkboxes can have an "indeterminate" state (a horizontal dash instead of a checkmark). This cannot be set via HTML; it must be set via JavaScript.

JavaScript

```
const checkbox = document.getElementById('myCheckbox');
checkbox.indeterminate = true;
```

3. Switches

Switches are a specialized version of checkboxes that look like a toggle button. Simply add the `.form-switch` class to the `.form-check` wrapper.

HTML

```
<div class="form-check form-switch">
  <input class="form-check-input" type="checkbox"
id="flexSwitchCheckDefault">
  <label class="form-check-label" for="flexSwitchCheckDefault">Toggle this
switch</label>
</div>
```

4. Inline Layout

By default, checkboxes and radios stack vertically. To place them on the same horizontal line, add the `.form-check-inline` class to the wrapper.

HTML

```
<div class="form-check form-check-inline">
  <input class="form-check-input" type="checkbox" id="inlineCheckbox1"
value="option1">
  <label class="form-check-label" for="inlineCheckbox1">1</label>
</div>
<div class="form-check form-check-inline">
  <input class="form-check-input" type="checkbox" id="inlineCheckbox2"
value="option2">
  <label class="form-check-label" for="inlineCheckbox2">2</label>
</div>
```

5. Reverse Alignment

If you want the checkbox or radio to appear on the **right** side of the label, add the `.form-check-reverse` class to the wrapper.

HTML

```
<div class="form-check form-check-reverse">
  <input class="form-check-input" type="checkbox" value=""
id="reverseCheck1">
  <label class="form-check-label" for="reverseCheck1">
    Reverse checkbox
  </label>
</div>
```

6. Toggle Buttons (Button Style)

You can make a checkbox or radio look like a standard Bootstrap button by using the `.btn-check` class on the input and standard button classes (like `.btn-outline-primary`) on the label.

HTML

```
<input type="checkbox" class="btn-check" id="btn-check" autocomplete="off">
<label class="btn btn-primary" for="btn-check">Checked</label>

<input type="radio" class="btn-check" name="options" id="option1"
autocomplete="off" checked>
<label class="btn btn-secondary" for="option1">Radio Button</label>
```

7. Disabled State

Add the `disabled` attribute to the `<input>` to gray out both the input and its associated label.

HTML

```
<div class="form-check">
  <input class="form-check-input" type="checkbox" value=""
id="flexCheckDisabled" disabled>
  <label class="form-check-label" for="flexCheckDisabled">
    Disabled checkbox
  </label>
</div>
```

Bootstrap 5 Range

Range inputs in Bootstrap 5 replace the default browser slider with a custom-styled, cross-browser consistent component. They are perfect for settings where a user needs to select a value from a numerical range, such as volume, brightness, or price filters.

1. Basic Range

To style a range input, add the `.form-range` class to an `<input type="range">`.

HTML

```
<label for="customRange1" class="form-label">Example range</label>
<input type="range" class="form-range" id="customRange1">
```

2. Min, Max, and Step

By default, range inputs have a range of 0 to 100 and move in increments of 1. You can customize this using standard HTML attributes:

- **min**: The lowest value.
- **max**: The highest value.
- **step**: The increment value (e.g., 0.5 for half-steps).

HTML

```
<label for="customRange3" class="form-label">Step range (0 to 5, steps of
0.5)</label>
<input type="range" class="form-range" min="0" max="5" step="0.5"
id="customRange3">
```

3. Disabled State

Add the `disabled` attribute to the input to make it non-interactive. This will also gray out the slider track and thumb to indicate it is inactive.

HTML

```
<label for="disabledRange" class="form-label">Disabled range</label>
<input type="range" class="form-range" id="disabledRange" disabled>
```

4. Styling the Track and Thumb

Bootstrap 5 uses CSS variables to style the range component. While there aren't many "built-in" modifier classes for color, you can easily override them in your CSS:

Variable

```
--bs-form-range-track-bg
--bs-form-range-thumb-bg
--bs-form-range-thumb-focus-box-shadow
```

Export to Sheets

Purpose

The color of the slider track.
The color of the circular slider handle.
The shadow color when the slider is clicked/focused.

5. Getting the Value (JavaScript)

Since the range input doesn't show the numerical value to the user by default, it is common practice to use a small bit of JavaScript to display the current selection in a label or `<span>`.

HTML

```
<label for="rangeInput" class="form-label">Volume: <span  
id="valueDisplay">50</span></label>  
<input type="range" class="form-range" id="rangeInput"  
oninput="valueDisplay.innerText = this.value">
```

6. Sizing and Layout

Range inputs work seamlessly within Bootstrap's layout utilities. You can place them inside **Input Groups** or use **Grid** columns to align them with other form controls.

HTML

```
<div class="row align-items-center">  
  <div class="col-auto">  
    <i class="bi bi-volume-down"></i>  
  </div>  
  <div class="col">  
    <input type="range" class="form-range">  
  </div>  
  <div class="col-auto">  
    <i class="bi bi-volume-up"></i>  
  </div>  
</div>
```

Bootstrap 5 Input Groups

Input Groups in Bootstrap 5 allow you to extend form controls by adding text, buttons, or button groups on either side of an input field. They are ideal for creating search bars, currency fields, or prefixing usernames.

1. Basic Structure

An input group is created by wrapping an input and its "add-on" inside a `<div>` with the `.input-group` class. Use `.input-group-text` for the labels or symbols.

HTML

```
<div class="input-group mb-3">
  <span class="input-group-text" id="basic-addon1">@</span>
  <input type="text" class="form-control" placeholder="Username" aria-label="Username">
</div>

<div class="input-group mb-3">
  <input type="text" class="form-control" placeholder="Recipient's username"
  aria-label="Recipient's username">
  <span class="input-group-text" id="basic-addon2">@example.com</span>
</div>
```

2. Sizing

You can change the size of the entire input group (including all its children) using modifier classes on the `.input-group` container.

- **Large:** `.input-group-lg`
 - **Small:** `.input-group-sm`
-

3. Using Checkboxes and Radios

You can place Bootstrap's custom checkboxes or radios inside an input group instead of text.

HTML

```
<div class="input-group mb-3">
  <div class="input-group-text">
    <input class="form-check-input mt-0" type="checkbox" value="" aria-label="Checkbox for following text input">
  </div>
  <input type="text" class="form-control" aria-label="Text input with checkbox">
</div>
```

4. Multiple Inputs and Add-ons

Input groups support multiple inputs in a single row, or multiple add-ons on the same side.

- **Multiple Add-ons:** Just place two `.input-group-text` elements side by side.

- **Multiple Inputs:** Place two `.form-control` elements inside. They will share the available width equally.

HTML

```
<div class="input-group">
  <span class="input-group-text">$</span>
  <span class="input-group-text">0.00</span>
  <input type="text" class="form-control" aria-label="Dollar amount (with dot
and two decimal places)">
</div>
```

5. Buttons and Dropdowns

You can attach full Bootstrap buttons or dropdown menus to your inputs. This is the standard way to build a "Search" bar with a button.

HTML

```
<div class="input-group mb-3">
  <button class="btn btn-outline-secondary" type="button" id="button-
addon1">Search</button>
  <input type="text" class="form-control" placeholder="Search keywords..."
aria-describedby="button-addon1">
</div>

<div class="input-group mb-3">
  <button class="btn btn-outline-secondary dropdown-toggle" type="button"
data-bs-toggle="dropdown" aria-expanded="false">Options</button>
  <ul class="dropdown-menu">
    <li><a class="dropdown-item" href="#">Action</a></li>
    <li><a class="dropdown-item" href="#">Another action</a></li>
  </ul>
  <input type="text" class="form-control" aria-label="Text input with
dropdown button">
</div>
```

6. Custom Selects and File Inputs

Input groups aren't just for text. You can include `.form-select` or file inputs as well.

HTML

```
<div class="input-group mb-3">
  <label class="input-group-text" for="inputGroupSelect01">Options</label>
  <select class="form-select" id="inputGroupSelect01">
    <option selected>Choose...</option>
    <option value="1">One</option>
  </select>
</div>
```

7. Accessibility

When using input groups, always ensure your inputs are accessible:

- Use `aria-label` or `aria-labelledby` on the input field, as the visual "add-on" may not be programmatically linked to the input by default.
- If the add-on provides the only label for the field, use an `id` on the add-on and `aria-describedby` on the input.

Bootstrap 5 Form Floating Labels

Floating Labels in Bootstrap 5 provide a modern, sleek interaction where the label rests inside the input field like a placeholder and then "floats" above the text once the user starts typing or the field gains focus.

1. Basic Structure

To create a floating label, wrap your `<input>` and `<label>` inside a `<div>` with the `.form-floating` class.

Crucial Rules:

1. The `<input>` **must** come before the `<label>`.
2. The `<input>` **must** have a `placeholder` attribute (even if it's an empty string `placeholder=" "`). Bootstrap uses the placeholder to detect when to float the label.

HTML

```
<div class="form-floating mb-3">
  <input type="email" class="form-control" id="floatingInput"
placeholder="name@example.com">
  <label for="floatingInput">Email address</label>
</div>
<div class="form-floating">
  <input type="password" class="form-control" id="floatingPassword"
placeholder="Password">
  <label for="floatingPassword">Password</label>
</div>
```

2. Textareas

Floating labels also work with `<textarea>` elements. By default, they will have the same height as an input. To increase the height, use custom CSS or inline styles—do not use the `rows` attribute, as it is often ignored by the floating label's height calculation.

HTML

```
<div class="form-floating">
  <textarea class="form-control" placeholder="Leave a comment here"
  id="floatingTextarea2" style="height: 100px"></textarea>
  <label for="floatingTextarea2">Comments</label>
</div>
```

3. Select Menus

Floating labels are also compatible with `.form-select`. Unlike text inputs, the label for a select menu **always** floats in the top position because the select already has a visible value.

HTML

```
<div class="form-floating">
  <select class="form-select" id="floatingSelect" aria-label="Floating label
  select example">
    <option selected>Open this select menu</option>
    <option value="1">One</option>
    <option value="2">Two</option>
  </select>
  <label for="floatingSelect">Works with selects</label>
</div>
```

4. Read-only and Plain Text

If you use the `readonly` attribute on an input, the floating label will still function. However, if you use `.form-control-plaintext` (which removes the borders), the floating label effect is usually disabled or looks inconsistent.

5. Layout with Grid

Floating labels work perfectly within the Bootstrap Grid. You can place multiple `.form-floating` containers inside columns to create side-by-side fields.

HTML

```
<div class="row g-2">
  <div class="col-md">
```

```
<div class="form-floating">
  <input type="email" class="form-control" id="floatingInputGroup"
placeholder="name@example.com" value="m@example.com">
  <label for="floatingInputGroup">Email address</label>
</div>
</div>
<div class="col-md">
  <div class="form-floating">
    <select class="form-select" id="floatingSelectGrid">
      <option selected>Open this select menu</option>
      <option value="1">One</option>
    </select>
    <label for="floatingSelectGrid">Works with selects</label>
  </div>
</div>
</div>
```

6. Important Limitations

- **Input Groups:** Floating labels are **not supported** inside `.input-group`. If you try to use them together, the labels will overlap with the `input-group-text` add-ons.
- **Type="file":** Floating labels do not support `<input type="file">` because that input type already has its own specific browser rendering for the "Choose File" button.

Bootstrap 5 Form Validation

Bootstrap 5 Form Validation provides a consistent, cross-browser way to provide feedback to users when they submit forms. It uses CSS pseudo-classes (`:invalid` and `:valid`) along with custom Bootstrap classes to style inputs and display feedback messages.

1. How it Works

Bootstrap offers two main ways to handle validation:

1. **Browser Defaults:** Uses the standard browser tooltips.
2. **Custom Styles (Recommended):** Uses custom CSS and JavaScript to provide a polished look that matches your site's theme.

To use custom styles, you must add the `.needs-validation` class to your `<form>` and include the `novalidate` attribute to disable default browser tooltips.

2. Basic Structure

Each form field should have a unique `id`, a corresponding `<label>`, and feedback containers.

HTML

```
<form class="row g-3 needs-validation" novalidate>
  <div class="col-md-4">
    <label for="validationCustom01" class="form-label">First name</label>
    <input type="text" class="form-control" id="validationCustom01"
value="Mark" required>
    <div class="valid-feedback">Looks good!</div>
  </div>
  <div class="col-md-4">
    <label for="validationCustomUsername" class="form-label">Username</label>
    <div class="input-group has-validation">
      <span class="input-group-text">@</span>
      <input type="text" class="form-control" id="validationCustomUsername"
required>
      <div class="invalid-feedback">Please choose a username.</div>
    </div>
  </div>
  <div class="col-12">
    <button class="btn btn-primary" type="submit">Submit form</button>
  </div>
</form>
```

3. Required JavaScript

To trigger the visual styles when a user clicks "Submit," you need a small script to check the form's validity and apply the `.was-validated` class.

JavaScript

```
(() => {
  'use strict'
  const forms = document.querySelectorAll('.needs-validation')

  Array.from(forms).forEach(form => {
    form.addEventListener('submit', event => {
      if (!form.checkValidity()) {
        event.preventDefault()
        event.stopPropagation()
      }
      form.classList.add('was-validated')
    }, false)
  })
})()
```

4. Validation Feedback Types

Bootstrap provides two styles for feedback text:

- **Standard:** Uses `.valid-feedback` or `.invalid-feedback`. These appear as plain text below the input.
- **Tooltips:** Uses `.valid-tooltip` or `.invalid-tooltip`. These appear as floating bubbles. To use these, the parent container must have `position: relative`.

HTML

```
<div class="col-md-4 position-relative">
  <input type="text" class="form-control" id="validationTooltip01" required>
  <div class="invalid-tooltip">Please provide a valid name.</div>
</div>
```

5. Supported Elements

Validation styles can be applied to almost all form controls:

- `<input>` (Text, Checkbox, Radio, File)
- `<select>`
- `<textarea>`

Note: For Checkboxes and Radios, ensure the feedback `div` is placed *inside* the `.form-check` wrapper for proper alignment.

6. Server-Side Validation

If you validate data on your server (e.g., checking if a username is taken), you can manually apply the following classes to your inputs without using the JavaScript "submit" listener:

- `.is-valid`: Forces the green "success" state.
- `.is-invalid`: Forces the red "error" state.

HTML

```
<input type="text" class="form-control is-invalid" id="user">
<div class="invalid-feedback">This username is already taken.</div>
```

7. Key Classes Summary

Class	Effect
-------	--------

Class	Effect
<code>.needs-validation</code>	Applied to <code><form></code> to enable custom Bootstrap styles.
<code>.was-validated</code>	Applied to <code><form></code> via JS to show feedback globally.
<code>.is-valid</code> / <code>.is-invalid</code>	Applied directly to inputs for manual/server-side status.
<code>.valid-feedback</code>	Text shown when the input is correct.
<code>.invalid-feedback</code>	Text shown when the input is incorrect.

Export to Sheets

Bootstrap 5 Grid System

The **Bootstrap 5 Grid System** is a mobile-first, 12-column system built with Flexbox. It allows you to create complex layouts that automatically adjust based on the screen size (breakpoints).

1. How it Works

The grid is composed of three main layers:

1. **Containers:** The outer wrapper that centers your content and adds horizontal padding.
2. **Rows:** Wrappers for columns. They ensure your columns are aligned correctly and negate the padding of the container.
3. **Columns:** The actual content holders. Columns must be immediate children of rows.

HTML

```
<div class="container">
  <div class="row">
    <div class="col">Column 1</div>
    <div class="col">Column 2</div>
    <div class="col">Column 3</div>
  </div>
</div>
```

2. Grid Options (Breakpoints)

Bootstrap uses six default breakpoints to handle responsiveness. You apply these by adding the abbreviation to the column class: `.col-{breakpoint}-{size}`.

Breakpoint	Class Abbr.	Dimensions
Extra small	None	< 576px
Small	sm	≥ 576px
Medium	md	≥ 768px

Breakpoint	Class Abbr.	Dimensions
Large	lg	≥ 992px
Extra large	xl	≥ 1200px
Extra extra large	xxl	≥ 1400px
Export to Sheets		

3. Auto-Layout Columns

- **Equal Width:** If you use just `.col`, Bootstrap will automatically divide the space equally between all columns in that row.
- **Setting One Width:** You can set the width of one column (e.g., `.col-6`), and the siblings with `.col` will automatically split the remaining space.
- **Variable Width Content:** Use `.col-auto` to make a column only as wide as its content.

HTML

```
<div class="row">
  <div class="col">Auto</div>
  <div class="col-6">Fixed 50% width</div>
  <div class="col">Auto</div>
</div>
```

4. Responsive Classes

You can define different widths for different screen sizes. For example, a column can take up the full width on mobile but only half the width on desktop.

HTML

```
<div class="row">
  <div class="col-12 col-md-6">Column A</div>
  <div class="col-12 col-md-6">Column B</div>
</div>
```

5. Gutters (Spacing)

Gutters are the padding between your columns, used to responsively space and align content.

- **Horizontal Gutters:** `.gx-0` to `.gx-5`.
- **Vertical Gutters:** `.gy-0` to `.gy-5`.
- **All Sides:** `.g-0` to `.g-5`.

6. Alignment

Since the grid is Flexbox-based, you can align columns inside a row easily.

- **Vertical Alignment:** Use `.align-items-start`, `.align-items-center`, or `.align-items-end` on the `.row`.
- **Horizontal Alignment:** Use `.justify-content-start`, `.justify-content-center`, or `.justify-content-between` on the `.row`.

HTML

```
<div class="row align-items-center justify-content-center" style="height: 200px;">
  <div class="col-4">I am perfectly centered!</div>
</div>
```

7. Nesting

To nest your content, add a new `.row` and set of `.col` elements inside an existing column.

HTML

```
<div class="container">
  <div class="row">
    <div class="col-sm-9">
      Level 1: .col-sm-9
      <div class="row">
        <div class="col-8 col-sm-6">Level 2: .col-8 .col-sm-6</div>
        <div class="col-4 col-sm-6">Level 2: .col-4 .col-sm-6</div>
      </div>
    </div>
  </div>
</div>
```

Bootstrap 5 Grid Stacked to horizontal

In Bootstrap 5, **Stacked-to-Horizontal** refers to the default responsive behavior of the grid. Because Bootstrap is **mobile-first**, columns start by stacking vertically on small screens and only move to a horizontal layout once a specific "breakpoint" (screen width) is reached.

1. How It Works

To create this effect, you use a breakpoint class like `.col-sm`, `.col-md`, or `.col-lg`.

- **Below the breakpoint:** The columns are 100% wide and stack on top of each other.
- **At and above the breakpoint:** The columns sit side-by-side.

HTML

```
<div class="container">
  <div class="row">
    <div class="col-md-8">.col-md-8 (Main Content)</div>
    <div class="col-md-4">.col-md-4 (Sidebar)</div>
  </div>

  <div class="row mt-3">
    <div class="col-sm">Equal width</div>
    <div class="col-sm">Equal width</div>
    <div class="col-sm">Equal width</div>
  </div>
</div>
```

2. Choosing the Right Breakpoint

The class you choose determines exactly when the layout "snaps" from vertical to horizontal.

Class	Breakpoint	Behavior
<code>.col-*</code>	None	Always horizontal (even on the smallest phones).
<code>.col-sm-*</code>	$\geq 576\text{px}$	Stacks on extra-small phones, horizontal on small phones+.
<code>.col-md-*</code>	$\geq 768\text{px}$	Most common. Stacks on phones, horizontal on tablets+.
<code>.col-lg-*</code>	$\geq 992\text{px}$	Stacks on phones/tablets, horizontal on laptops+.
<code>.col-xl-*</code>	$\geq 1200\text{px}$	Stacks until reaching a large desktop monitor.

Export to Sheets

3. Mixing and Matching

You can define multiple behaviors for the same element. For example, you might want a column to be full-width on mobile, half-width on tablets, and one-third width on large desktops.

HTML

```
<div class="row">
  <div class="col-12 col-md-6 col-lg-4">Card 1</div>
  <div class="col-12 col-md-6 col-lg-4">Card 2</div>
  <div class="col-12 col-md-6 col-lg-4">Card 3</div>
</div>
```

4. Why use `.col-12`?

In Bootstrap 5, `div` elements are block-level, so they naturally stack. However, adding `.col-12` is a "best practice" when mixing breakpoints (like the example above) to explicitly state that on the smallest screens, the column should occupy all 12 units of the row.

5. Horizontal Alignment (Flexbox)

If your columns are horizontal but don't fill the whole 12-column row, you can use flex utilities on the `.row` to position them:

- **Center them:** `.justify-content-center`
- **Push them apart:** `.justify-content-between`
- **Align to the end:** `.justify-content-end`

HTML

```
<div class="row justify-content-center">
  <div class="col-md-4">Centered horizontally on desktop</div>
</div>
```

Bootstrap 5 Grid Extra Small

In Bootstrap 5, the **Extra Small (xs)** grid tier applies to the smallest screen sizes, typically vertical mobile phones with a width of **less than 576px**.

Unlike other breakpoints, the Extra Small tier does not have a specific abbreviation like `sm` or `md`. Instead, you use the basic `.col-*` classes.

1. How it Works

The `.col-*` classes are **global**. Because Bootstrap is mobile-first, a class like `.col-6` applies to Extra Small devices and *all* larger devices unless a higher breakpoint (like `.col-md-4`) overrides it.

HTML

```
<div class="container">
  <div class="row">
    <div class="col-6">50% Width</div>
```

```
<div class="col-6">50% Width</div>
</div>
</div>
```

2. Default Stacking

If you do not specify a number (e.g., just using `.col-md-6`), the columns will default to **100% width** on Extra Small screens. They will stack vertically until the screen hits the `md` (768px) threshold.

To prevent stacking on the smallest phones, you **must** use the basic `.col-*` classes.

HTML

```
<div class="row">
  <div class="col-md-6">I stack on mobile</div>
  <div class="col-md-6">I stack on mobile</div>
</div>

<div class="row">
  <div class="col-6">I stay side-by-side</div>
  <div class="col-6">I stay side-by-side</div>
</div>
```

3. Mixing XS with Larger Breakpoints

The most powerful way to use the XS tier is to define a mobile layout that changes as the screen grows.

HTML

```
<div class="row">
  <div class="col-12 col-md-4">Column 1</div>
  <div class="col-12 col-md-4">Column 2</div>
  <div class="col-12 col-md-4">Column 3</div>
</div>
```

4. Auto-Width on XS

You can use `.col-auto` on Extra Small screens to make a column only as wide as its content. This is useful for small icons or labels next to a flexible text input.

HTML

```
<div class="row">
  <div class="col-auto">
```

```
<button class="btn btn-primary">Fixed</button>
</div>
<div class="col">
  <input type="text" class="form-control" placeholder="I take the rest of
the space">
</div>
</div>
```

5. Summary Table

Breakpoint	Class	Width
Extra Small (xs)	.col-*	< 576px
Small (sm)	.col-sm-*	≥ 576px
Medium (md)	.col-md-*	≥ 768px
Export to Sheets		

Pro-Tip: If you want a 3-column layout on mobile, use `.col-4`. If you want a 2-column layout, use `.col-6`. If you use `.col`, they will simply split the 12-column space equally (e.g., 3 columns become 4 units each).

Bootstrap 5 Grid Small

In Bootstrap 5, the **Small (sm)** grid tier is the first responsive breakpoint above the default (extra small). It targets devices with a screen width of **576px or wider**, such as most smartphones in landscape mode or smaller tablets.

1. The sm Breakpoint

The `.col-sm-*` classes kick in once the viewport hits **576px**.

- **Below 576px:** The columns will automatically stack vertically (100% width).
- **At 576px and above:** The columns will align horizontally based on the number of units you defined.

HTML

```
<div class="container">
  <div class="row">
    <div class="col-sm-6">Column 1</div>
    <div class="col-sm-6">Column 2</div>
  </div>
</div>
```

2. Auto-Layout for Small

If you don't want to calculate the 12-column math, you can use `.col-sm`. This will make the columns share the available space equally, but **only** once the screen is at least 576px wide.

HTML

```
<div class="row">
  <div class="col-sm">1 of 3 (Stacked < 576px)</div>
  <div class="col-sm">2 of 3 (Stacked < 576px)</div>
  <div class="col-sm">3 of 3 (Stacked < 576px)</div>
</div>
```

3. Mixing Small with Other Tiers

Because Bootstrap is mobile-first, a `.col-sm-*` class applies to the small, medium, large, and extra-large tiers **unless** a larger breakpoint overrides it.

You can create a "fluid" layout that changes at every major device size:

HTML

```
<div class="row">
  <div class="col-12 col-sm-6 col-lg-4">Content Block</div>
</div>
```

4. Small Grid Alignment

You can use flexbox utility classes on the `.row` to change how "small" columns are positioned. These can also be responsive!

- `.justify-content-sm-center`: Centers the columns only on small screens and up.
 - `.align-items-sm-end`: Aligns columns to the bottom only on small screens and up.
-

5. Summary Table for Small Tier

Feature	Details
Breakpoint	≥ 576px
Class Prefix	<code>.col-sm-</code>

Feature	Details
Container Max-width	540px
Gutter Width	1.5rem (24px) by default
Export to Sheets	

Pro-Tip: If your layout looks "broken" on a standard iPhone or Android held vertically, it's likely because you are using `.col-sm-*` and the screen is narrower than 576px. Switch to `.col-*` (Extra Small) if you want the side-by-side look on vertical phones.

Bootstrap 5 Grid Medium

In Bootstrap 5, the **Medium (md)** grid tier is the most commonly used breakpoint. It targets devices with a screen width of **768px or wider**, which typically includes tablets (like an iPad) in portrait mode or small laptops.

1. The `md` Breakpoint

The `.col-md-*` classes activate once the viewport hits **768px**.

- **Below 768px (Mobile/Small Devices):** The columns will automatically stack vertically (100% width) unless you have defined an `xs` (`.col-*`) or `sm` (`.col-sm-*`) class.
- **At 768px and above (Tablets/Laptops):** The columns will align horizontally based on the 12-column system.

HTML

```
<div class="container">
  <div class="row">
    <div class="col-md-8">Main Content Area</div>
    <div class="col-md-4">Sidebar</div>
  </div>
</div>
```

2. Default Stacking vs. Horizontal

Because Bootstrap is **mobile-first**, if you only use `.col-md-6`, the browser assumes that on anything smaller than a tablet, the content should take up the full width.

Screen Width	Class Used: <code>.col-md-6</code>	Class Used: <code>.col-6</code>
< 768px (Phone)	Stacked (100% width)	Horizontal (50% width)
≥ 768px (Tablet)	Horizontal (50% width)	Horizontal (50% width)
Export to Sheets		

3. Mixing Breakpoints for Fluidity

To create a professional, responsive layout, you often mix the `md` tier with `sm` or `lg` to control how the "snap" happens across different devices.

HTML

```
<div class="row">
  <div class="col-12 col-md-6 col-lg-3">Box 1</div>
  <div class="col-12 col-md-6 col-lg-3">Box 2</div>
  <div class="col-12 col-md-6 col-lg-3">Box 3</div>
  <div class="col-12 col-md-6 col-lg-3">Box 4</div>
</div>
```

4. Medium Tier Container

When you use a standard `.container`, the maximum width of your content changes as the screen grows. At the medium breakpoint:

- **Screen Width:** $\geq 768\text{px}$
 - **Max Content Width:** **720px**
 - **Class Prefix:** `.col-md-`
-

5. Responsive Vertical Alignment

You can use the `md` prefix with flexbox utilities to change alignment **only** on medium screens and up. This is great for centering content on a tablet while letting it stay at the top on a phone.

HTML

```
<div class="row align-items-md-center" style="height: 200px;">
  <div class="col">I'm centered on my iPad, but at the top on my
  iPhone.</div>
</div>
```

Bootstrap 5 Grid Large

In Bootstrap 5, the **Large (lg)** grid tier is designed for standard desktop monitors and laptops. It targets devices with a screen width of **992px or wider**. At this breakpoint, you typically have enough horizontal space to move from a stacked or simple tablet layout to a more complex multi-column desktop layout.

1. The `lg` Breakpoint

The `.col-lg-*` classes activate once the viewport hits **992px**.

- **Below 992px:** The columns will stack vertically (100% width) unless you have defined smaller breakpoints like `md`, `sm`, or `xs`.
- **At 992px and above:** The columns will align horizontally according to the 12-column grid.

HTML

```
<div class="container">
  <div class="row">
    <div class="col-lg-3">Sidebar Left</div>
    <div class="col-lg-6">Main Content Area</div>
    <div class="col-lg-3">Sidebar Right</div>
  </div>
</div>
```

2. Container Behavior at Large

When using the standard `.container` class, Bootstrap sets a specific `max-width` to ensure your content doesn't stretch too far across a wide monitor, which can make text hard to read.

Feature	Details
Breakpoint	≥ 992px
Class Prefix	<code>.col-lg-</code>
Container Max-width	960px
Export to Sheets	

3. Responsive Scaling (Fluidity)

A common professional pattern is to increase the number of columns as the screen size increases. Using `lg` in combination with `md` and `sm` allows for a seamless transition.

HTML

```
<div class="row">
  <div class="col-12 col-md-6 col-lg-3">Product 1</div>
  <div class="col-12 col-md-6 col-lg-3">Product 2</div>
  <div class="col-12 col-md-6 col-lg-3">Product 3</div>
  <div class="col-12 col-md-6 col-lg-3">Product 4</div>
</div>
```

4. Large-Specific Utilities

You can use the `lg` prefix with almost any Bootstrap utility to apply styles **only** on desktop screens. This is incredibly useful for hiding or showing elements based on the device.

- `.d-lg-block`: Shows an element only on large screens and up.
- `.d-none .d-lg-block`: Hides an element on mobile/tablet but shows it on desktop.
- `.text-lg-end`: Aligns text to the right only on large screens.

5. Row Columns for Large

Instead of adding classes to every single `div`, you can set the column count on the `.row` itself using `.row-cols-lg-*`.

HTML

```
<div class="row row-cols-1 row-cols-md-2 row-cols-lg-4">
  <div class="col">Item 1</div>
  <div class="col">Item 2</div>
  <div class="col">Item 3</div>
  <div class="col">Item 4</div>
</div>
```

Bootstrap 5 Grid Extra Large

In Bootstrap 5, the **Extra Large (xl)** grid tier is designed for large desktop monitors and high-resolution screens. It targets devices with a screen width of **1200px or wider**. This tier is often used to refine layouts that might feel too "stretched out" on smaller desktop views (the `lg` tier).

1. The `xl` Breakpoint

The `.col-xl-*` classes activate once the viewport hits **1200px**.

- **Below 1200px**: The columns will follow the rules of the `lg`, `md`, or `sm` classes you've defined. If none are defined, they stack vertically.
- **At 1200px and above**: The columns align horizontally according to the 12-column grid.

HTML

```
<div class="container">
```

```
<div class="row">
  <div class="col-12 col-md-6 col-lg-4 col-xl-3">Product Card 1</div>
  <div class="col-12 col-md-6 col-lg-4 col-xl-3">Product Card 2</div>
</div>
</div>
```

2. Container Behavior at Extra Large

When using the standard `.container` class, the width of your content is capped to maintain optimal readability. At the extra-large breakpoint:

Feature	Details
Breakpoint	≥ 1200px
Class Prefix	<code>.col-xl-</code>
Container Max-width	1140px
Export to Sheets	

3. XL Specific Utilities

The `xl` prefix works with almost all Bootstrap utility classes. This allows you to create highly specific desktop experiences that differ from mobile or tablet views.

- **Spacing:** `.px-xl-5` (Add large horizontal padding only on XL screens).
 - **Display:** `.d-xl-none` (Hide a sidebar or element only on XL screens to simplify the view).
 - **Flexbox:** `.justify-content-xl-between` (Spread items out only when there is enough width).
-

4. Row Columns (Shorthand)

If you have a gallery of many items, you can use `.row-cols-xl-*` on the parent row to set the layout for all child columns at once.

HTML

```
<div class="row row-cols-1 row-cols-md-3 row-cols-xl-6">
  <div class="col">Item 1</div>
  <div class="col">Item 2</div>
  <div class="col">Item 3</div>
  <div class="col">Item 4</div>
  <div class="col">Item 5</div>
  <div class="col">Item 6</div>
</div>
```

5. Why Use XL?

While the `lg` breakpoint (992px) covers most laptops, the `xl` breakpoint is essential for:

- **Complex Dashboards:** Providing more room for data tables or multiple sidebars.
- **Photography Portfolios:** Showing more images in a single row without them becoming too small.
- **E-commerce:** Displaying more products per row on large iMacs or external monitors.

Bootstrap 5 Grid XXL

The **Extra Extra Large (xxl)** grid tier was introduced in Bootstrap 5 to accommodate the increasing number of ultra-wide monitors and high-resolution displays (like 4K screens or 27-inch iMacs). It targets devices with a screen width of **1400px or wider**.

1. The `xxl` Breakpoint

The `.col-xxl-*` classes activate once the viewport hits **1400px**. On screens smaller than this, the columns will follow the rules of the `xl`, `lg`, `md`, or `sm` classes you have defined.

HTML

```
<div class="container">
  <div class="row">
    <div class="col-12 col-md-6 col-lg-3 col-xxl-2">Gallery Item 1</div>
    <div class="col-12 col-md-6 col-lg-3 col-xxl-2">Gallery Item 2</div>
    <div class="col-12 col-md-6 col-lg-3 col-xxl-2">Gallery Item 3</div>
    <div class="col-12 col-md-6 col-lg-3 col-xxl-2">Gallery Item 4</div>
    <div class="col-12 col-md-6 col-lg-3 col-xxl-2">Gallery Item 5</div>
    <div class="col-12 col-md-6 col-lg-3 col-xxl-2">Gallery Item 6</div>
  </div>
</div>
```

2. Container Behavior at XXL

The standard `.container` class reaches its maximum fixed width at this breakpoint. This prevents your content from becoming "too wide," which can lead to awkwardly long lines of text that are difficult to read.

Feature

Details

Feature	Details
Breakpoint	$\geq 1400\text{px}$
Class Prefix	<code>.col-xxl-</code>
Container Max-width	1320px
Export to Sheets	

3. XXL Specific Utilities

The `xxl` prefix can be used with almost all Bootstrap utility classes (spacing, display, flex, etc.). This allows you to fine-tune the layout for users with massive screen real estate.

- **Spacing:** `.m-xxl-5` (Apply maximum margin only on ultra-wide screens).
 - **Display:** `.d-none` `.d-xxl-flex` (Show a specific layout component only on XXL screens).
 - **Grid Gutters:** `.g-xxl-5` (Increase the space between columns on very large screens to make the design feel more "airy").
-

4. Row Columns for XXL

If you are building a product grid or a photo gallery, use `.row-cols-xxl-*` to quickly set how many items appear per row on ultra-wide displays.

HTML

```
<div class="row row-cols-1 row-cols-lg-4 row-cols-xxl-8">
  <div class="col">Item 1</div>
  <div class="col">Item 2</div>
</div>
```

5. Why the XXL Tier Matters

Before XXL was added, a design that looked perfect on a 13-inch laptop (1280px) might look "stretched" or have too much empty space on a 27-inch monitor. The XXL tier allows you to:

- **Add more columns:** Move from 4 columns to 6 or 8 columns.
- **Increase font sizes/padding:** Scale up the UI elements to match the larger physical screen size.

- **Complex Multi-pane Layouts:** Perfect for "Dashboard" style apps where you want to show a sidebar, a main content area, and a secondary details panel all at once.

6. Grid Summary Table (All Breakpoints)

Tier	Abbreviation	Breakpoint	Container Width
Extra Small	None	< 576px	100%
Small	sm	≥ 576px	540px
Medium	md	≥ 768px	720px
Large	lg	≥ 992px	960px
Extra Large	xl	≥ 1200px	1140px
Extra Extra Large	xxl	≥ 1400px	1320px

Export to Sheets

Bootstrap 5 Grid Examples

1. Equal Width Columns (Auto-Layout)

If you don't specify a number (1-12), Bootstrap automatically divides the space equally. This works on all screen sizes.

HTML

```
<div class="container">
  <div class="row">
    <div class="col">1 of 3</div>
    <div class="col">2 of 3</div>
    <div class="col">3 of 3</div>
  </div>
</div>
```

2. Setting One Column Width

You can set one column to a specific width (e.g., 6 units for 50%), and the remaining .col siblings will automatically split the rest of the space.

HTML

```
<div class="container">
  <div class="row">
    <div class="col">Auto-split</div>
    <div class="col-6">Fixed 50% Width</div>
    <div class="col">Auto-split</div>
  </div>
```

</div>

3. Responsive "Stack to Horizontal"

This is the most common example for a **Main Content + Sidebar** layout. It stacks on mobile and goes side-by-side on tablets (`md`).

HTML

```
<div class="container">
  <div class="row">
    <div class="col-md-8">Main Article Content</div>
    <div class="col-md-4">Sidebar / Related Links</div>
  </div>
</div>
```

4. Mixing All Breakpoints (The "Fluid" Grid)

Use this for a product gallery. It shows 1 item on phones, 2 on tablets, and 4 on large desktops.

HTML

```
<div class="container">
  <div class="row">
    <div class="col-12 col-md-6 col-lg-3">Product Card 1</div>
    <div class="col-12 col-md-6 col-lg-3">Product Card 2</div>
    <div class="col-12 col-md-6 col-lg-3">Product Card 3</div>
    <div class="col-12 col-md-6 col-lg-3">Product Card 4</div>
  </div>
</div>
```

5. Centering a Column (Offsetting)

If you want a 50% width box centered in the middle of the screen, you use an offset of 3 (since $3+6+3=12$).

HTML

```
<div class="container">
  <div class="row">
    <div class="col-md-6 offset-md-3">
      I am a centered 50% width box on medium screens.
    </div>
  </div>
</div>
```

6. Vertical and Horizontal Alignment

You can center content both ways using Flexbox utilities on the `.row`.

HTML

```
<div class="container">
  <div class="row align-items-center justify-content-center" style="min-height:
300px;">
    <div class="col-4 border">
      Perfectly Centered Content
    </div>
  </div>
</div>
```

7. Row Columns (Shorthand)

Instead of adding classes to every single column `div`, you can set the "cols per row" on the parent `.row`.

HTML

```
<div class="row row-cols-2 row-cols-lg-4 g-4">
  <div class="col"><div class="p-3 border">Item 1</div></div>
  <div class="col"><div class="p-3 border">Item 2</div></div>
  <div class="col"><div class="p-3 border">Item 3</div></div>
  <div class="col"><div class="p-3 border">Item 4</div></div>
</div>
```

Summary of Logic

- `.container`: Centers your layout.
- `.row`: Holds the columns and negates padding.
- `.col-[breakpoint]-[1-12]`: Defines the width at specific screen sizes.
- **Total = 12**: Your column numbers in a single row should ideally add up to 12.